BTS SIO · OPTION SISR · ÉPREUVE E6

Homelab sur la baie Proxmox

Documentation technique

Infrastructure de stockage et de services : virtualisation, conteneurs, accès distant, supervision, sécurité et sauvegarde

Auteur	Axel Royer

Cadre	BTS SIO option SISR — réalisation professionnelle n°1 (épreuve E6)
Version	1.3 — 3 novembre 2026
État	Phases 0 à 5 et 7 terminées ; phase 6 presque terminée (accès du jury à finaliser) ; phase 8 (Bookaru) à venir

Sommaire

Synthèse du projet 3

1. Contexte et objectifs 4

2. Besoin et choix techniques 5

2.1 Besoin et contraintes 5
2.2 Choix techniques 5

3. Architecture 7

3.1 Vue d'ensemble 7
3.2 Inventaire des composants 7
3.3 Services, ports et dossiers partagés 7

4. Mise en œuvre 10

4.1 Virtualisation, système et stockage 10
4.2 Docker et Portainer 11
4.3 Jellyfin et partage SMB 12
4.4 Tableau de bord Homepage 12

5. Accès distant 14

5.1 Tailscale et adresse unique 14
5.2 Jellyfin et OMV en HTTPS 15

6. Supervision 17

7. Sécurité 19

7.1 Comptes et privilèges 19
7.2 SSH en root désactivé et traçabilité 20
7.3 Isolation de la VM dans le tailnet 21
7.4 Accès du jury (à finaliser) 22

8. Sauvegarde et restauration 23

8.1 Snapshots Proxmox 23
8.2 Sauvegarde vzdump 24
8.3 Test de restauration 26
8.4 Objectifs de reprise (RPO et RTO) 29

9. Exploitation 30

9.1 Procédures courantes 30
9.2 Dépannage rapide 30
9.3 Mises à jour 31

10. Incidents rencontrés et enseignements 32

11. Tests de validation 33

12. Scénario de démonstration (10 minutes) 34

13. Limites et perspectives 36

Annexe A. Fichiers Docker Compose 37

Annexe B. Configuration de Homepage (services.yaml) 38

Annexe C. Politique d'accès Tailscale 39

Annexe D. Commandes utiles 39

Synthèse du projet

Objectif. Héberger, sur une machine virtuelle de la baie Proxmox de l'école, des services de stockage et des applications conteneurisées, administrables à distance de façon sécurisée, supervisés et sauvegardés.

Ce qui a été réalisé

- Une VM OpenMediaVault 8.3.1 (Debian 13) sur le VLAN 11, avec quatre services Docker : Portainer, Jellyfin, Uptime Kuma et Homepage.
- Un accès unique par l'adresse 172.31.11.10, à l'école comme à distance (Tailscale), et des interfaces en HTTPS avec un certificat Let's Encrypt.
- Une supervision avec alertes Discord par service, validée par des pannes provoquées.
- Une administration sécurisée : compte nominatif et `sudo`, connexion root en SSH désactivée, VM isolée du reste du réseau privé par une étiquette.
- Une sauvegarde hebdomadaire sur le stockage `qnap`, avec une restauration testée vers une VM de test.

Tableau 1 — Chiffres clés et preuves

Indicateur	Valeur	Section
Panne d'un service (test)	Alerte Down à 11:49:08, Up à 11:49:28 : 20 s d'indisponibilité	6
Mémoire disponible de la VM	4,7 Gio sur 6 Gio	4.1
Certificat HTTPS	Let's Encrypt, valable 90 jours (2 nov. 2026 au 31 janv. 2027)	5.2
Isolation de la VM	Connexion vers le NAS personnel : OUVERT avant l'étiquette, BLOQUE après	7.3
Sauvegarde de la VM en service	7 min 43 s, archive de 4,59 Go (84 Gio parcourus, 88 % de zones vides)	8.2
Restauration vers une VM de test	7 min 20 s	8.3

État d'avancement et limites. Les phases 0 à 5 et 7 sont terminées ; la phase 6 (sécurité) l'est presque, il reste l'accès du jury ; l'application Bookaru (phase 8) est à venir. Limites principales : la supervision tourne sur la VM qu'elle surveille, le stockage de sauvegarde est celui de l'école, et le démarrage de la VM restaurée n'a pas été testé.

Où trouver quoi. Architecture : section 3 · Choix techniques : section 2 · Objectifs de reprise : section 8.4 · Scénario de démonstration de 10 minutes : section 12 · Fichiers de configuration : annexes.

1. Contexte et objectifs

Ce document décrit la conception, la mise en œuvre et l'exploitation d'une infrastructure de services hébergée sur une machine virtuelle de la baie Proxmox de l'école. Elle reproduit, en version réduite, un homelab personnel fondé sur **OpenMediaVault** (stockage, partages, administration) et **Docker** (services applicatifs).

Objectifs

- Héberger plusieurs services sur une VM unique : médiathèque (Jellyfin), administration de conteneurs (Portainer), supervision (Uptime Kuma) et tableau de bord (Homepage).
- Accéder à la VM avec **une seule adresse**, depuis le réseau de l'école comme à distance.
- Superviser les services et être alerté en cas de panne (notifications Discord, un salon par service).
- Sécuriser l'administration : comptes nominatifs, connexion SSH en root désactivée, chiffrement HTTPS, isolation de la VM dans le réseau privé virtuel.
- Sauvegarder la VM sur un stockage séparé et **prouver** que la restauration fonctionne.

Démarche. Le projet est découpé en phases validées une à une. Avant chaque manipulation sensible, un snapshot Proxmox est pris pour pouvoir revenir en arrière. Chaque incident rencontré est consigné avec sa cause et sa correction (section 10).

Tableau 2 — Avancement par phase (au 3 novembre 2026)

Phase	Contenu	État
P0-1	Infrastructure Proxmox, installation d'OpenMediaVault, réseau, stockage	Fait
P2	omv-extras, plugin Compose, Docker, Portainer	Fait
P3	Jellyfin et partage SMB des films	Fait
P4	Supervision : Uptime Kuma, alertes Discord, Homepage	Fait
P5	Accès distant : Tailscale, route 172.31.11.10/32, Jellyfin en HTTPS	Fait
P6	Sécurité : comptes, SSH, HTTPS d'OMV, isolation par étiquette	Presque terminée : accès du jury à finaliser avec l'école
P7	Sauvegarde vzdump vers qnap et test de restauration	Fait
P8	Application Bookaru (Caddy, exposition publique par Tailscale Funnel)	À venir

2. Besoin et choix techniques

2.1 Besoin et contraintes

Le projet répond à un besoin précis : héberger plusieurs services accessibles en toute sécurité depuis l'école et à distance, sur une infrastructure partagée dont on ne maîtrise ni le réseau ni l'hyperviseur.

Tableau 3 — Besoins et réponses apportées

Besoin	Exigence	Réponse apportée	Section
Héberger des services	Plusieurs applications sur une VM modeste	Docker Compose et Portainer	4.2
Partager des fichiers	Dépôt depuis Windows, lecture par les applications	OMV, partage SMB, Jellyfin en lecture seule	4.3
Accéder partout	Même adresse à l'école et à distance, sans ouvrir de port	Tailscale, route annoncée `/32`	5.1
Chiffrer	Interfaces d'administration en HTTPS	Tailscale Serve, certificat Let's Encrypt	5.2
Savoir quand un service tombe	Alertes distinctes par service	Uptime Kuma et Discord	6
Limiter les risques	Moindre privilège, isolation	Comptes nominatifs, SSH, étiquette	7
Ne pas perdre la VM	Copie séparée et restaurable	Sauvegarde vzdump, test de restauration	8

Contraintes

- Une VM de 2 vCPU et 6 Go de RAM (4,7 Gio disponibles) : tous les services doivent y tenir.
- Une baie partagée avec les autres élèves : accès limité à sa propre VM (pool `axel.r`), aucune modification du réseau.
- Le VLAN 11 n'est pas joignable depuis le poste par le VPN de l'école (voir 5.1).
- Aucune exposition du service sur Internet à ce stade.
- Des solutions libres ou gratuites, reproductibles : tout est décrit par des fichiers (Compose, YAML).

2.2 Choix techniques

Tableau 4 — Choix retenus et alternatives écartées

Domaine	Choix retenu	Alternatives	Raison du choix
Stockage et partages	OpenMediaVault	TrueNAS ; Samba configuré à la main	Léger (Debian), interface web, plugin Compose ; TrueNAS repose sur ZFS, plus gourmand en mémoire ; Samba seul n'apporte pas d'administration
Services	Docker Compose avec Portainer	Installation native ; Kubernetes	Isolation, reproductibilité et mises à jour simples ; Kubernetes est disproportionné pour une VM
Médiathèque	Jellyfin	Plex	Libre et utilisable sans compte externe
Accès distant	Tailscale (WireGuard)	VPN auto-hébergé (OpenVPN, WireGuard)	Aucun port à ouvrir sur le réseau de l'école, ACL et certificats HTTPS inclus ; un VPN auto-hébergé demande un serveur joignable et la gestion des clés

Domaine	Choix retenu	Alternatives	Raison du choix
Supervision	Uptime Kuma	Zabbix	Sondes simples et alertes en quelques minutes ; Zabbix mesure davantage mais exige serveur, base de données et agents
Alertes	Discord (webhooks)	Courrier électronique ; SMS	Notification immédiate sur mobile, un salon par service, sans serveur de messagerie à configurer
Tableau de bord	Homepage	Heimdall, Homarr	Lit l'état des conteneurs par le socket Docker ; configuration en fichiers YAML
Sauvegarde	vzdump vers `qnap`	Proxmox Backup Server ; copie manuelle	Outil intégré à Proxmox et stockage fourni ; Proxmox Backup Server (déduplication, incrémental) demande un serveur dédié
Contrôle d'accès	Comptes nominatifs et étiquette Tailscale	Compte root partagé ; pas d'isolation	Traçabilité et moindre privilège ; empêche le rebond de la VM vers un autre appareil

Ces choix privilégient la simplicité et la légèreté, au prix de limites assumées (section 13).

3. Architecture

3.1 Vue d'ensemble

Le schéma d'ensemble (figure 1, en fin de section) montre les deux réseaux en jeu : le **réseau de l'école** (baie Proxmox, VLAN 11) et un **réseau privé virtuel Tailscale** (tailnet) qui relie le poste d'administration à la VM. La VM appartient aux deux : elle porte l'adresse 172.31.11.10 sur le réseau de l'école et une adresse Tailscale dans le tailnet.

Lecture du schéma.

- Le poste d'administration atteint l'interface Proxmox par le VPN de l'école (machine `proxmox-acces`), puis la VM par Tailscale.
- La VM annonce dans le tailnet sa propre adresse (`172.31.11.10/32`) : la même adresse fonctionne donc à l'école et à distance.
- La VM est **étiquetée** `tag:ecole` : elle ne peut pas ouvrir de connexion vers le NAS personnel du même tailnet (voir 7.3).
- Les disques de la VM sont sur `local-lvm` ; une sauvegarde hebdomadaire est écrite sur `qnap`, stockage séparé géré par l'école.
- Les alertes de supervision partent vers Discord en HTTPS (webhooks).
- Le jury (accès prévu) rejoindrait la VM par un partage Tailscale, sans accès au reste du tailnet.

3.2 Inventaire des composants

Tableau 5 — Composants de l'infrastructure

Composant	Caractéristiques
Hyperviseur	Proxmox VE 9.2.2, cluster de 3 nœuds (pve-1, pve-2, pve-3) ; VM hébergée sur pve-1, pool `axel.r`
Machine virtuelle	VM 107 `VM-Homelab-axel` : 2 vCPU (1 socket, 2 cœurs), 6 Gio de RAM, machine q35, BIOS SeaBIOS, contrôleur VirtIO SCSI single, agent QEMU activé
Disques	`scsi0` 20 Go (système) et `scsi1` 64 Go (données, EXT4, `/dev/sdb1`), sur `local-lvm`
Réseau	Carte VirtIO sur `vmbr1`, VLAN 11, pare-feu activé ; 172.31.11.10/24, passerelle 172.31.11.254, DNS 1.1.1.1
Système	Debian 13 « trixie », noyau 7.2.11, OpenMediaVault 8.3.1, nom d'hôte `homelab-axel`
Conteneurs	Docker 29.9.2, Compose 5.6.1, plugin `openmediavault-compose` 8.2.5
Services	Portainer CE 2.45.3 (lts), Jellyfin, Uptime Kuma 2, Homepage
Accès distant	Tailscale, tailnet `tail14f35b.ts.net`, VM `homelab-axel` (100.101.158.79), étiquette `tag:ecole`
Sauvegarde	Tâche vzdump hebdomadaire de la VM 107 vers le stockage `qnap`

3.3 Services, ports et dossiers partagés

Tableau 6 — Services et adresses d'accès

Service	Adresse sur le réseau de l'école (ou route /32)	Adresse HTTPS via Tailscale
OpenMediaVault	http://172.31.11.10 (port 80)	https://homelab-axel.tail14f35b.ts.net:9444
Jellyfin	http://172.31.11.10:8096	https://homelab-axel.tail14f35b.ts.net:8443

Service	Adresse sur le réseau de l'école (ou route /32)	Adresse HTTPS via Tailscale
Portainer	https://172.31.11.10:9443 (certificat auto-signé)	—
Uptime Kuma	http://172.31.11.10:3001	—
Homepage	http://172.31.11.10:3000	—
Partage SMB `films`	\\172.31.11.10\films (port 445)	—
SSH	`ssh Axel@172.31.11.10` (port 22), puis `sudo`	—

Tableau 7 — Dossiers partagés OpenMediaVault (disque de données de 64 Go)

Dossier	Usage
`appdata`	Fichiers Compose des services (réglage « Compose Files »)
`dockerdata`	Données persistantes des conteneurs (réglage « Data », remplace `CHANGE_TO_COMPOSE_DATA_PATH`)
`dockerstore`	Stockage interne de Docker (images, couches), réglage « Docker storage »
`films`	Médias lus par Jellyfin, déposés par partage SMB

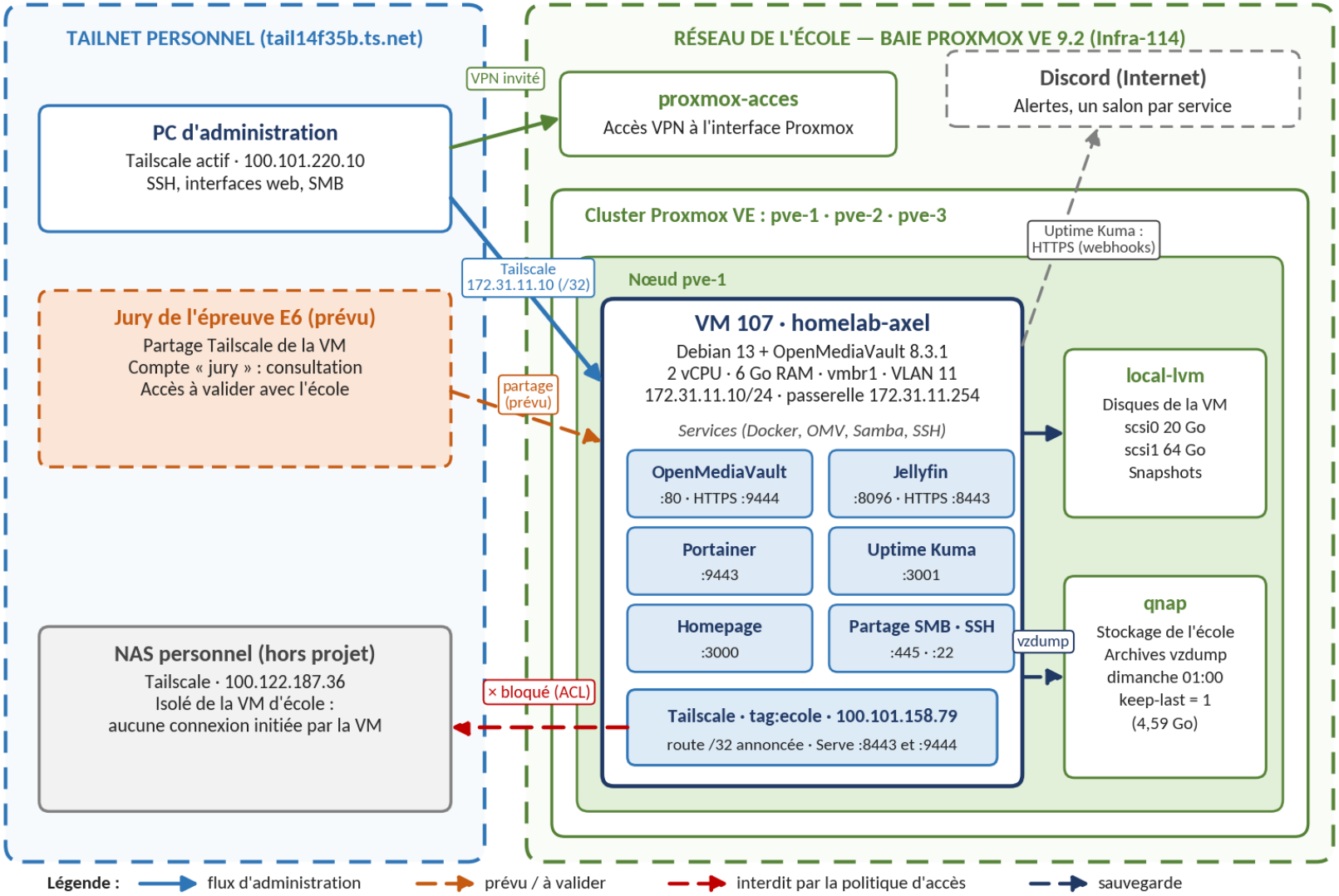


***Figure 1** — Architecture de l'infrastructure : réseaux, VM 107, services, stockage et flux*

4. Mise en œuvre

4.1 Virtualisation, système et stockage

La VM 107 a été créée sur le nœud pve-1 à partir de l'image d'installation d'OpenMediaVault 8.3.1 (stockée sur `qnap`). Le processeur est de type générique pour que la VM reste déplaçable dans le cluster. L'adresse IP est statique sur le VLAN 11 ; la validation s'est faite par ping de la passerelle, accès à Internet et résolution DNS depuis la VM.

Le disque de données de 64 Go a été effacé, formaté en EXT4 et monté dans OMV. Les dossiers partagés `appdata` et `films` y ont été créés, puis `dockerdata` et `dockerstore`. L'agent QEMU a été installé dans la VM après un premier échec dû à une dépendance (`libnuma1`) : Proxmox remonte désormais l'adresse IP de la VM et sa mémoire réelle.

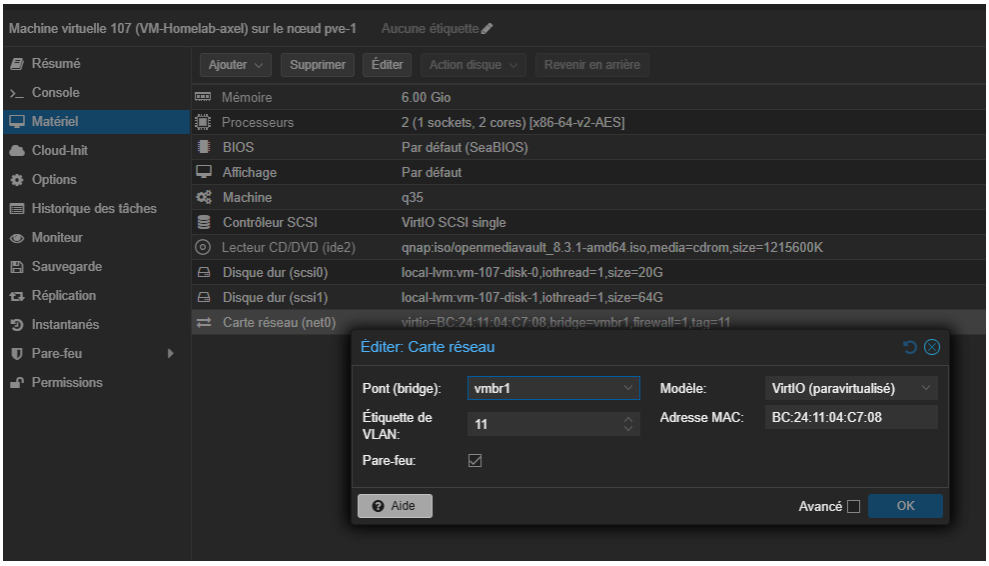


Figure 2 — *Matériel de la VM 107 : mémoire, processeurs, disques et carte réseau (vmbr1, VLAN 11)*

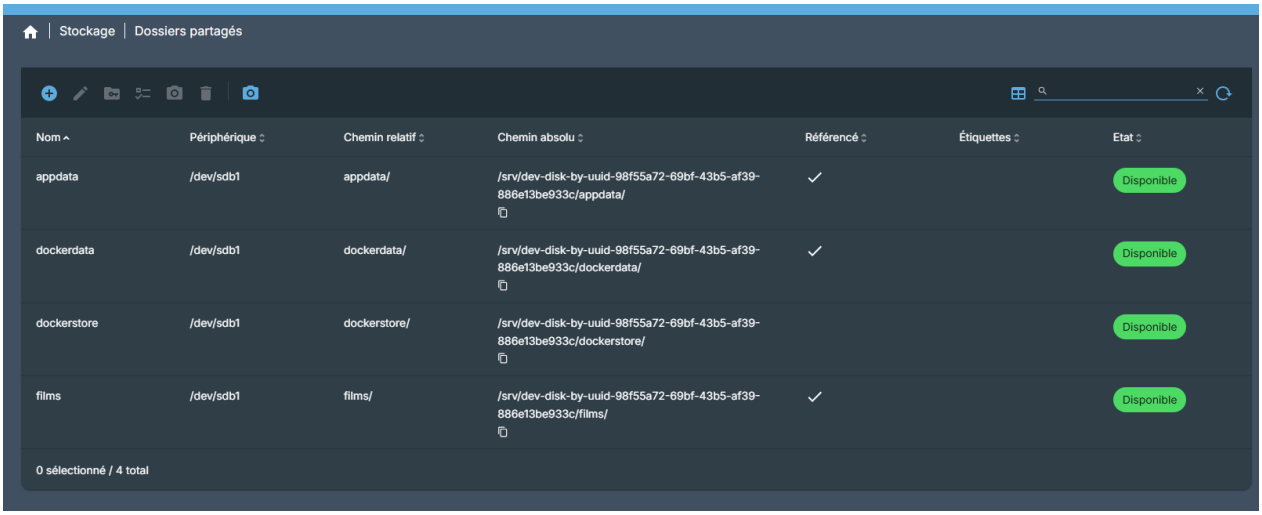


Figure 3 — *Dossiers partagés d'OMV sur le disque de données /dev/sdb1 : appdata, dockerdata, dockerstore et films*

> **Mémoire.** Avant l'agent, Proxmox affichait 91 % de RAM utilisée alors que `free -h` montrait 1,0 Gio utilisé et 4,7 Gio disponibles : la différence était du cache disque, que Linux libère à la demande. La mémoire « utilisée » vue par l'hyperviseur n'est pas la mémoire disponible pour les applications.

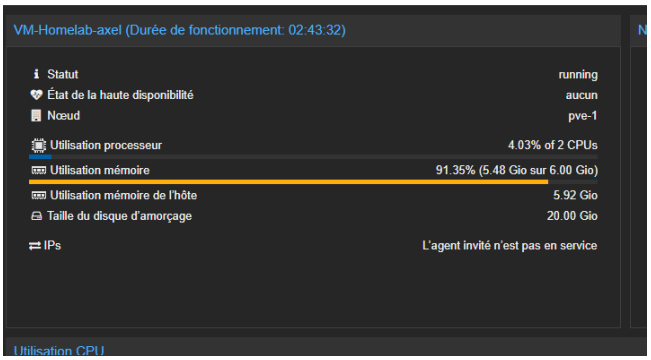


Figure 4 — Résumé Proxmox avant l'installation de l'agent : 91 % de mémoire affichée et « l'agent invité n'est pas en service »

4.2 Docker et Portainer

Docker est installé par omv-extras et le plugin Compose. Les chemins sont réglés dans Services → Compose → Paramètres : fichiers Compose dans `appdata`, données des conteneurs dans `dockerdata`, stockage Docker dans `dockerstore`. La commande `docker info | grep "Docker Root Dir"` confirme que Docker écrit sur le disque de 64 Go et non sur le disque système.

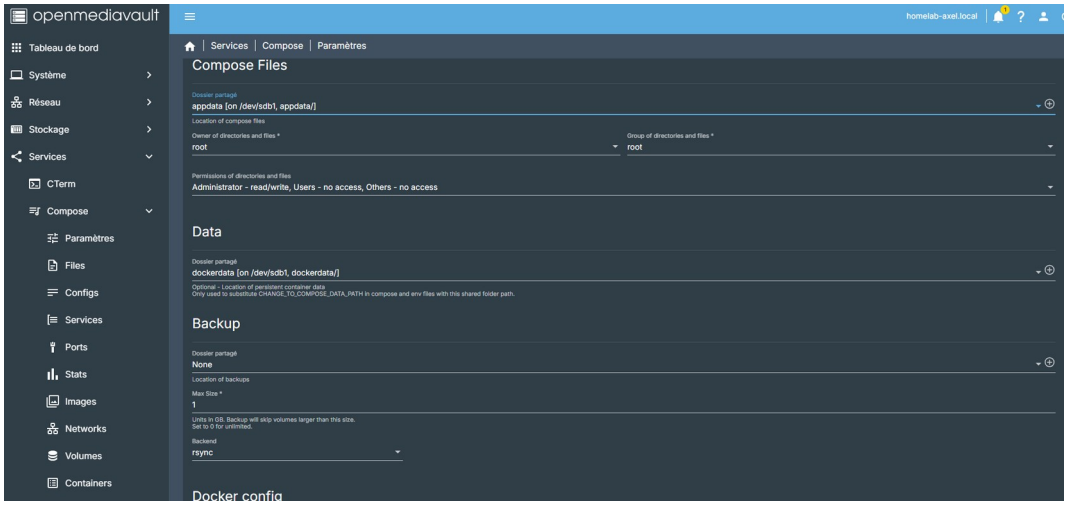


Figure 5 — Réglages Compose d'OMV : dossiers « Compose Files » (appdata) et « Data » (dockerdata)

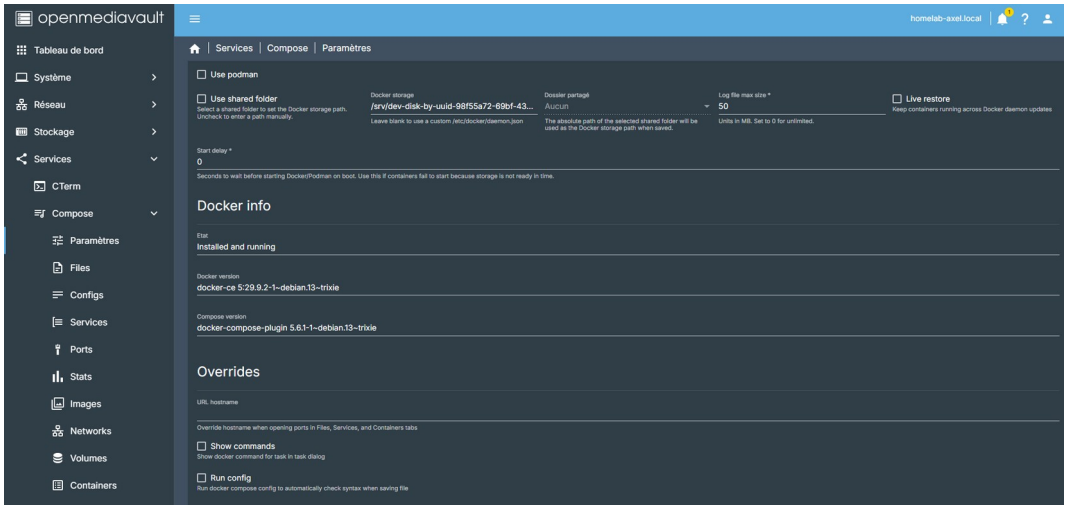


Figure 6 — Chemin de stockage Docker (dockerstore) et état du service : Docker 29.9.2 et Compose 5.6.1 installés

Portainer est déployé par un fichier Compose (annexe A). Trois points d'attention :

- enregistrer le fichier dans OMV ne démarre rien : il faut cliquer sur **Up** pour créer le conteneur ;

- cette version de Portainer exige un **jeton d'initialisation** à la première connexion, affiché dans les journaux (`docker logs portainer`) ;
- le compte administrateur doit être créé dans les 5 minutes qui suivent le démarrage, avec 12 caractères minimum.

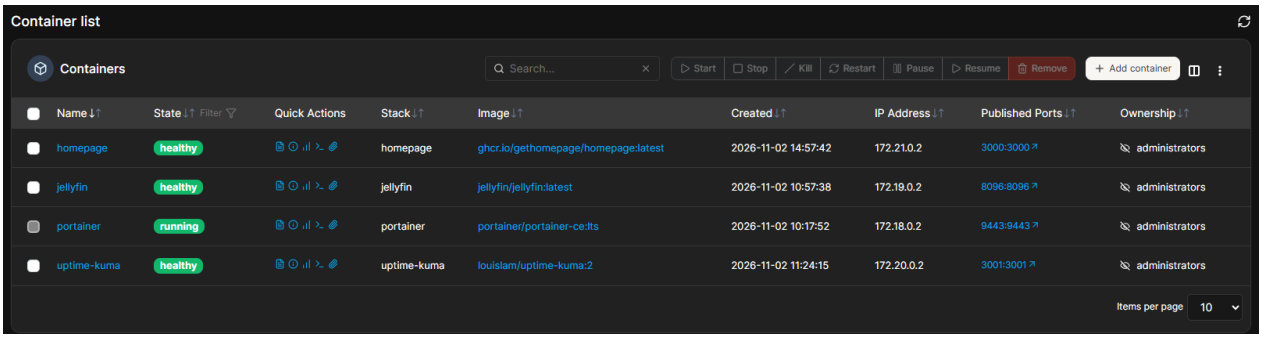


Figure 7 — Liste des conteneurs dans Portainer : quatre conteneurs, un réseau Docker par service, ports publiés

Chaque fichier Compose crée son propre réseau Docker (adresses 172.18.0.0 à 172.21.0.0) : les conteneurs ne communiquent pas entre eux par défaut, et seuls les ports publiés sont joignables depuis la VM.

4.3 Jellyfin et partage SMB

Un partage SMB sur le dossier `films` permet de déposer les médias depuis Windows (`\\172.31.11.10\films`). Il utilise un utilisateur OMV dédié disposant des droits de lecture et d'écriture sur le dossier. Jellyfin monte ce dossier en **lecture seule** (`/media/films`) : il peut lire les films mais ni les modifier ni les supprimer.

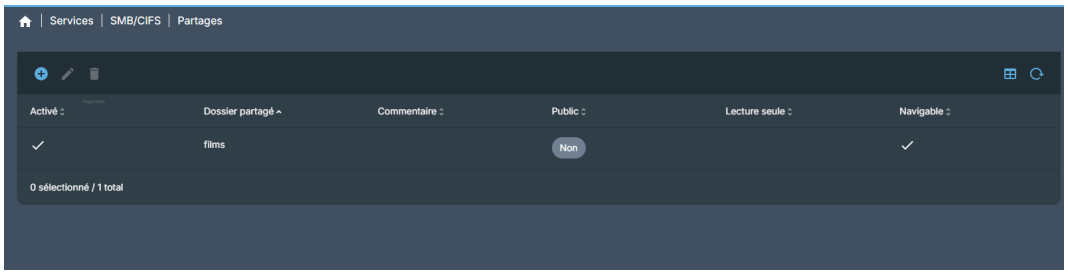


Figure 8 — Partage SMB « films » dans OMV : activé, navigable, non public, écriture autorisée

> **Incident : « erreur fatale » à la lecture.** Les journaux montraient ffprobe puis ffmpeg en échec. La cause : les films téléchargés auparavant faisaient **0 octet** (la commande `curl` n'était alors pas installée, seuls des fichiers vides avaient été créés). Correction : suppression des fichiers vides et nouveau téléchargement avec `curl -fL`, dont l'option `-f` fait échouer la commande sur erreur HTTP au lieu de créer un fichier vide. Le film « Tears of Steel » (film libre de la fondation Blender, 355 Mo) se lit correctement.

4.4 Tableau de bord Homepage

Homepage regroupe les services et affiche leur état. Il se configure par trois fichiers YAML dans `dockerdata/homepage` : `docker.yaml` (accès au socket Docker, pour l'état des conteneurs), `services.yaml` (les tuiles) et `widgets.yaml` (processeur, mémoire, disque, heure). La variable `HOMEPAGE_ALLOWED_HOSTS` est obligatoire : elle liste les adresses autorisées, toute autre est refusée.

Les tuiles OpenMediaVault et Jellyfin existent en deux versions : une en HTTPS (adresse du tailnet, poste avec Tailscale actif) et une de repli en HTTP sur 172.31.11.10, utilisable depuis un poste de l'école sans Tailscale. Un certificat valide s'obtient pour un nom et non pour une adresse IP, d'où ce doublon.

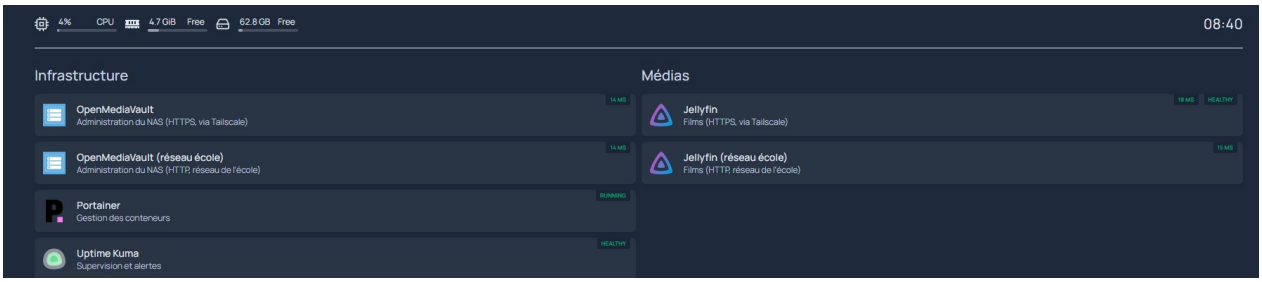


Figure 9 — *Tableau de bord Homepage : widgets système, services et état des conteneurs*

Incident : erreur YAML « bad indentation ». Un collage dans l'éditeur nano s'était ajouté à la suite du contenu existant, sans retour à la ligne (`docker.socklocal:`). Diagnostic avec `cat -A`, qui affiche les fins de ligne et les caractères cachés ; correction en vidant le fichier avant de coller. En YAML, l'indentation fait partie de la syntaxe.

5. Accès distant

5.1 Tailscale et adresse unique

L'accès à l'interface Proxmox passe par `proxmox-acces`, une machine du réseau privé de l'école partagée dans le tailnet. Un partage de machine Tailscale ne donne accès **qu'à cette machine**, jamais aux sous-réseaux qu'elle route : le VLAN 11 (172.31.11.0/24) était donc injoignable depuis le poste, en SSH comme sur les interfaces web. La console Proxmox restait le seul accès.

Diagnostic : depuis la console, la VM pingait sa passerelle, donc la VM était saine. Le problème se situait sur le chemin entre le poste et le VLAN 11. **Solution :** installer Tailscale directement sur la VM, dans le tailnet personnel de l'administrateur.

1. Vérifier SSH : `systemctl status ssh`.
2. Installer curl (absent par défaut) : `apt update && apt install -y curl`.
3. Installer Tailscale : `curl -fsSL https://tailscale.com/install.sh | sh`.
4. Joindre le tailnet : `tailscale up --hostname=homelab-axel`, puis valider l'URL de connexion dans le navigateur (état `NeedsLogin` tant que ce n'est pas fait).
5. Contrôler l'adresse : `tailscale ip -4` renvoie 100.101.158.79.

Une seule adresse partout. Avec le seul nom `homelab-axel`, les accès ne fonctionnent que sur un poste où Tailscale est actif. La VM annonce donc dans le tailnet une route réduite à sa propre adresse, puis la route est approuvée dans la console d'administration :

```
tailscale up --hostname=homelab-axel --advertise-routes=172.31.11.10/32
```

Les avertissements affichés (IP forwarding, UDP GRO sur `ens18`) sont sans conséquence ici : le forwarding ne sert que si la VM fait transiter du trafic vers un autre réseau, et le GRO est une optimisation de débit. **Validation :** `Test-NetConnection 172.31.11.10 -Port 22` renvoie `TcpTestSucceeded : True` avec l'interface `Tailscale`.

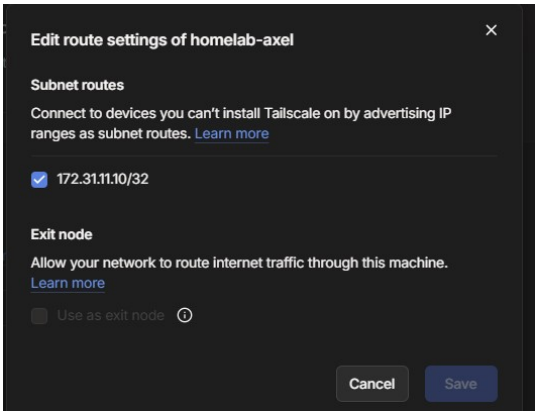


Figure 10 — Réglages de route de la machine homelab-axel : route 172.31.11.10/32 approuvée, aucun exit node

> **Expiration de clé.** Par défaut, la clé d'une machine expire : à l'échéance, le trafic relayé est interrompu jusqu'à une nouvelle authentification (la console l'avait signalé par l'avertissement « Key expiry is enabled » dans les réglages de route). Pour une VM qui doit rester joignable, l'expiration a été **désactivée** pour cette machine (console Tailscale, Machines, menu de la machine, « Disable key expiry »), ce que confirme le badge « Expiry disabled ». Contrepartie : la clé reste valable tant qu'elle n'est pas retirée ; si la VM était compromise, elle serait supprimée du tailnet à la main, et l'étiquette `tag:ecole` limite déjà ce qu'elle peut atteindre (voir 7.3).

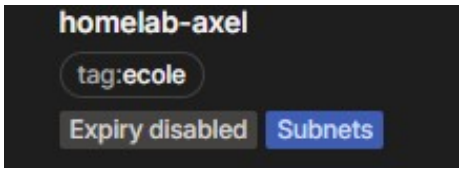


Figure 11 — La machine homelab-axel après la désactivation de l'expiration de clé : badges tag:ecole, Expiry disabled et Subnets

Tableau 8 — Comportement de l'adresse 172.31.11.10 selon la situation du poste

Situation du poste	Tailscale sur le poste	Résultat
À distance	Actif	Fonctionne par le tunnel (route approuvée)
Réseau de l'école	Coupé	Fonctionne en direct sur le réseau local
Réseau de l'école	Actif	Fonctionne aussi ; couper Tailscale évite un détour par le tunnel
À distance	Coupé	Ne fonctionne pas (adresse privée de l'école)

Limites : seule cette adresse est joignable, pas le reste du VLAN 11 (un `/32` désigne un seul hôte), et la VM doit être allumée et connectée à Tailscale.

5.2 Jellyfin et OMV en HTTPS

Les interfaces sont publiées en HTTPS avec **Tailscale Serve**, qui fournit un certificat Let's Encrypt pour le nom du tailnet et reste **privé** (accessible uniquement depuis le tailnet). Prérequis : MagicDNS et les certificats HTTPS activés dans la console d'administration.

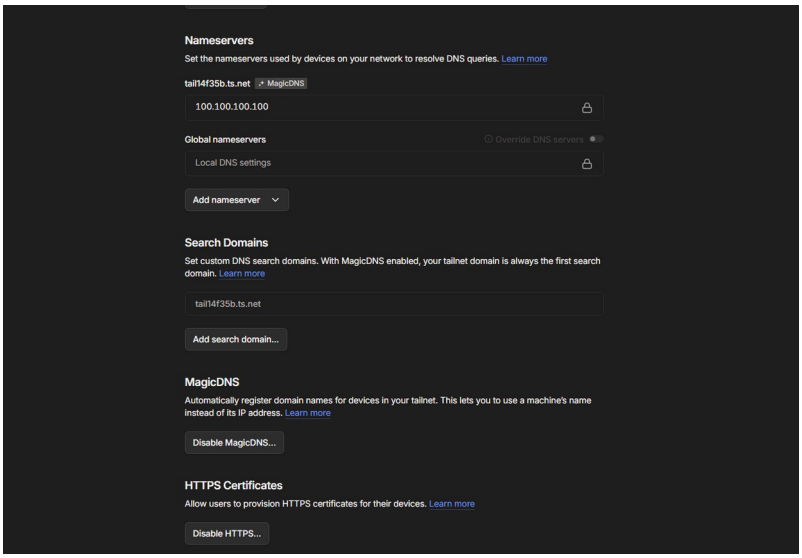


Figure 12 — Console Tailscale (DNS) : MagicDNS et certificats HTTPS activés, tailnet tail14f35b.ts.net

```
tailscale serve --bg --https=8443 http://127.0.0.1:8096   # Jellyfin
tailscale serve --bg --https=9444 http://127.0.0.1:80     # interface OpenMediaVault
```

`--bg` maintient la publication après un redémarrage. Les ports 8443 et 9444 ont été choisis pour laisser libre le **port 443**, réservé à Tailscale Funnel pour l'exposition publique de l'application Bookaru (phase 8) : Serve accepte n'importe quel port, alors que Funnel est limité aux ports 443, 8443 et 10000. Le HTTPS natif d'OMV a été écarté : certificat auto-signé et conflit avec le port 443.

```
root@homelab-axel:~# tailscale serve status
https://homelab-axel.tail14f35b.ts.net:8443 (tailnet only)
|-- / proxy http://127.0.0.1:8096

root@homelab-axel:~# echo | openssl s_client -connect 100.101.158.79:8443 -servername homelab-axel.tail14f35b.ts.net 2>/dev/nul
l | openssl x509 -noout -issuer -subject -dates
issuer=C=US, O=Let's Encrypt, CN=YE1
subject=CN=homelab-axel.tail14f35b.ts.net
notBefore=Nov  2 13:58:51 2026 GMT
notAfter=Jan 31 13:58:50 2027 GMT
root@homelab-axel:~#
```

Figure 13 — *Publications Serve (tailnet only) et certificat Let's Encrypt contrôlé avec openssl*

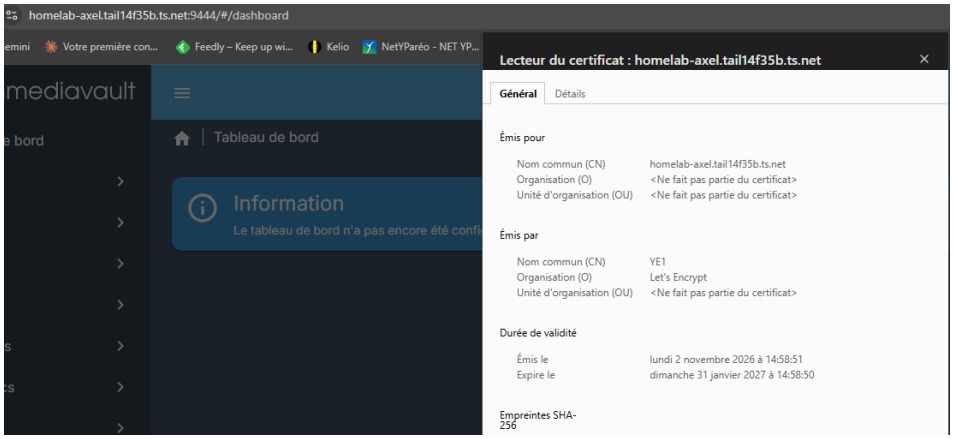


Figure 14 — *Interface d'OpenMediaVault en HTTPS (port 9444) et certificat émis pour le nom du tailnet*

Le certificat (Let's Encrypt, autorité intermédiaire YE1) est valable du 2 novembre 2026 au 31 janvier 2027 ; il est géré et renouvelé par Tailscale, et un seul certificat couvre tous les services publiés sur la machine.

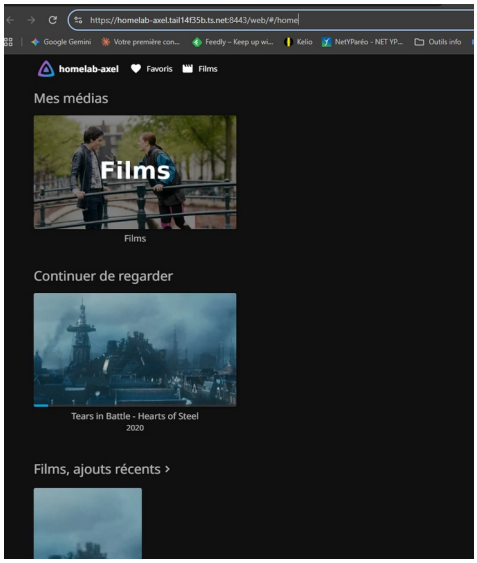


Figure 15 — *Jellyfin servi en HTTPS par Tailscale Serve : adresse du tailnet sur le port 8443 et médiathèque « Films »*

Tableau 9 — Apport et limites du HTTPS

Apport	Explication
Confidentialité	Mot de passe et jeton de session chiffrés (TLS), illisibles pour un tiers qui écouterait le réseau
Authentification du serveur	Le certificat prouve que l'on parle bien à `homelab-axel`
Intégrité	Les échanges ne peuvent pas être modifiés en route
Limite	Le tunnel Tailscale (WireGuard) chiffre déjà le trafic ; l'accès HTTP de repli reste actif à l'école et n'est pas protégé par ce chiffrement

6. Supervision

Uptime Kuma surveille trois services par des sondes HTTP(S). Les sondes ciblent l'adresse IP de la VM et non son nom, car le conteneur ne passe pas par la résolution MagicDNS. Réglage normal : intervalle de 60 s, 2 nouvelles tentatives espacées de 20 s, soit une alerte après trois échecs consécutifs, ce qui évite les fausses alertes.

Tableau 10 — Sondes de supervision

Sonde	Type	Cible
OMV	HTTP(s)	http://172.31.11.10
Jellyfin	HTTP(s)	http://172.31.11.10:8096/health
Portainer	HTTP(s)	https://172.31.11.10:9443 (erreurs TLS ignorées : certificat auto-signé)

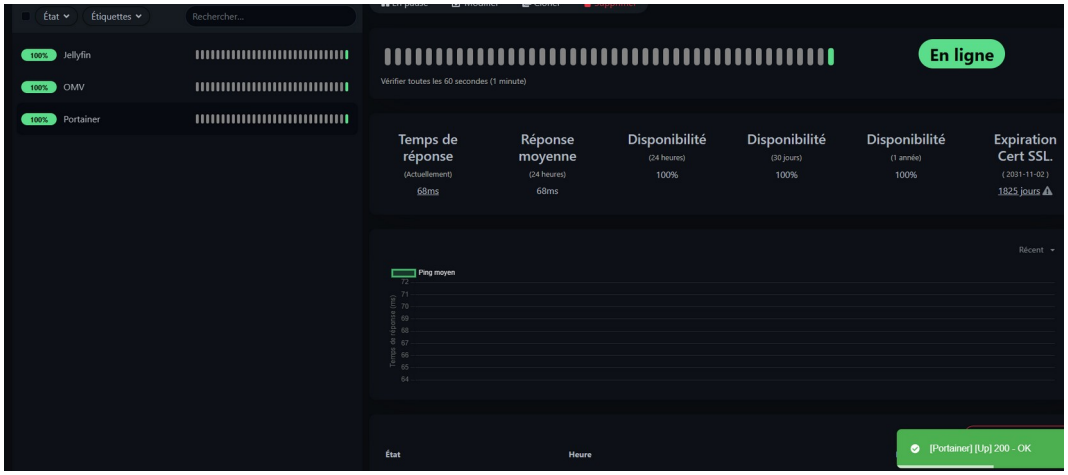


Figure 16 — Uptime Kuma : les trois sondes en ligne

Alertes Discord. Un serveur Discord dédié comporte un salon et un webhook par service (`#omv`, `#jellyfin`, `#portainer`). Dans Kuma, une notification Discord est créée par webhook et rattachée uniquement à la sonde correspondante. L'URL d'un webhook est un secret : elle n'est saisie que dans Kuma.

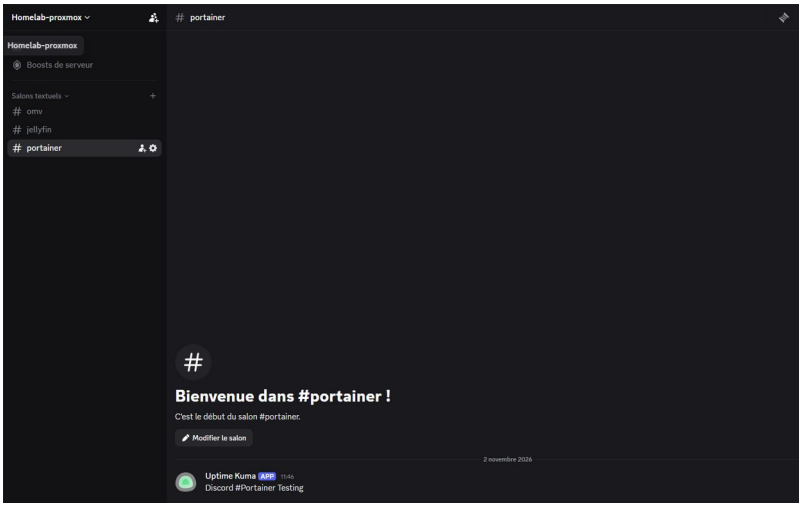


Figure 17 — Serveur Discord dédié : un salon par service ; le message de test envoyé par Uptime Kuma s'affiche dans #portainer

Tests de panne (intervalle de 20 s et aucune nouvelle tentative pendant le test, puis retour aux valeurs normales) :

Tableau 11 — Tests de panne validés

Service	Commande	Résultat
Jellyfin	`docker stop jellyfin` puis `docker start jellyfin`	Alerte Down à 11:49:08, Up à 11:49:28 (20 s d'indisponibilité), erreur ECONNREFUSED
Portainer	`docker stop portainer` puis `docker start portainer`	Alertes Down et Up dans `#portainer`
OMV	`monit unmonitor nginx`, `systemctl stop nginx`, puis `systemctl start nginx` et `monit monitor nginx`	Alertes Down et Up dans `#omv`

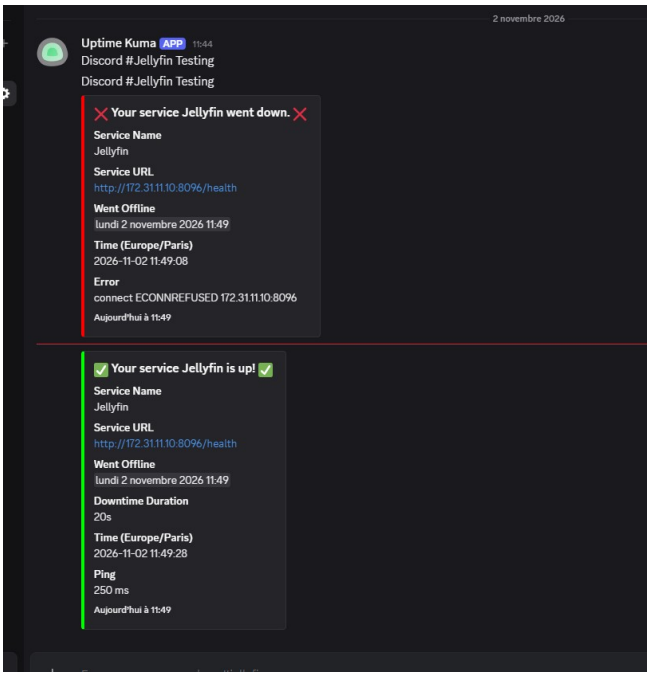


Figure 18 — *Alertes Down puis Up reçues dans le salon Discord du service*

ECONNREFUSED signifie que la VM répond mais que plus aucun processus n'écoute sur le port : c'est une panne du **service**, alors qu'une panne de la machine donnerait un délai d'attente dépassé. Sur OMV, **monit** relance nginx automatiquement : il faut suspendre sa surveillance le temps du test, ce qui illustre l'auto-réparation du système.

> **Limite assumée.** Uptime Kuma tourne sur la VM qu'il surveille : si la VM tombe, il ne peut rien signaler. En production, la supervision serait placée sur une machine distincte.

7. Sécurité

Cette phase répond à la question : que se passe-t-il si quelqu'un tente de prendre le contrôle de la VM ? Elle repose sur le principe du moindre privilège et sur la défense en profondeur : aucune mesure n'est suffisante seule, c'est leur accumulation qui rend une attaque difficile.

Tableau 12 — Mesures de sécurité et risques couverts

Mesure	Risque couvert
Compte nominatif avec `sudo`	Compte `root` partagé et sans traçabilité
Connexion SSH en `root` désactivée	Attaque par force brute sur un compte au nom connu
Mot de passe `admin` d'OMV changé dès la création	Identifiants par défaut, publics
Interfaces OMV et Jellyfin en HTTPS	Mots de passe et sessions lisibles sur le réseau
Isolation de la VM par étiquette Tailscale	Rebond de la VM vers un autre appareil du tailnet
Compte `jury` sans `sudo` ni `docker`	Élévation de privilèges par un accès de consultation
Sauvegarde sur un stockage séparé	Perte de la VM ou de son disque

7.1 Comptes et privilèges

Tableau 13 — Comptes Linux

Compte	Rôle	Groupes	Droits
`Axel` (UID 1000)	Administration et partage SMB (compte créé dans OMV)	users, sudo, _ssh	Administration par `sudo`, journalisée
`jury` (UID 1001)	Consultation par le jury	jury, users, _ssh	Connexion SSH uniquement ; ni `sudo` ni `docker`
`root`	Compte système	—	Connexion SSH désactivée

Sous OMV, un utilisateur doit appartenir au groupe `_ssh` pour se connecter en SSH. Le groupe `docker` est volontairement refusé à `Axel` comme à `jury` : il permet de lancer un conteneur monté sur le disque de l'hôte, donc d'obtenir en pratique les droits root. Les commandes Docker s'exécutent avec `sudo docker …`, ce qui les rend traçables.

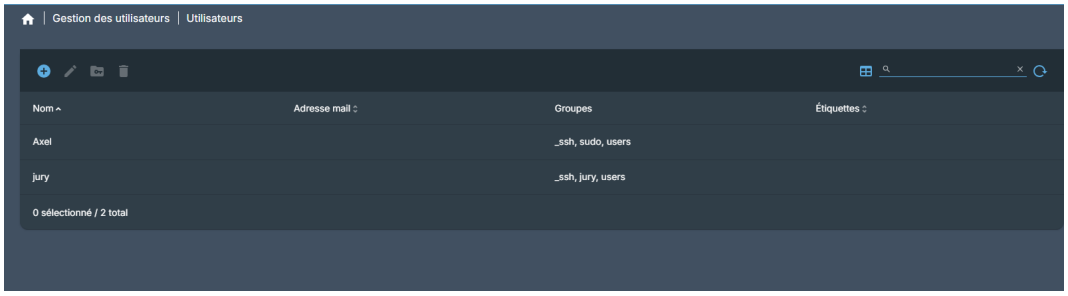


Figure 19 — Utilisateurs d'OMV : Axel (_ssh, sudo, users) et jury (_ssh, jury, users)

Incident : deux comptes qui ne diffèrent que par la casse. Le compte créé dans OMV pour le partage SMB s'appelle `Axel` (majuscule). Un compte `axel` (minuscule) avait été créé par erreur en ligne de commande : sous Linux, les noms sont sensibles à la casse et ce sont deux comptes distincts. Après vérification (liste des comptes, groupes, dossiers `/home`), `axel` a été supprimé et `Axel` conservé ; il a reçu le shell bash, les groupes `sudo` et `_ssh` par l'interface OMV (qui gère ces comptes), un dossier personnel (`mkhomedir_helper`) et des droits resserrés (`chmod 700`).

7.2 SSH en root désactivé et traçabilité

`root` existe sur toutes les machines Linux : son nom est connu de tout attaquant, qui n'a plus qu'à deviner un mot de passe. L'administration passe désormais par `Axel` et `sudo`. Le réglage se fait dans Services → SSH d'OMV, pour qu'il tienne lors des déploiements de configuration.

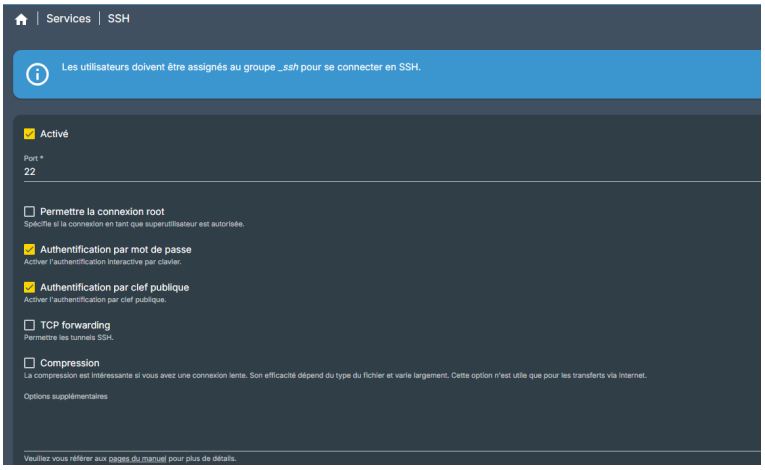


***Figure 20** — Paramètres SSH d'OMV : connexion root désactivée, mot de passe et clef publique actifs*

Tableau 14 — Validation de la désactivation du root

Contrôle	Résultat
`sshd -T | grep -i permitrootlogin` (configuration réellement chargée)	`permitrootlogin no`
`ssh root@172.31.11.10`	`Permission denied`, même avec le bon mot de passe
`ssh Axel@172.31.11.10` puis `sudo -i`	Fonctionne ; l'invite passe à `root@homelab-axel`

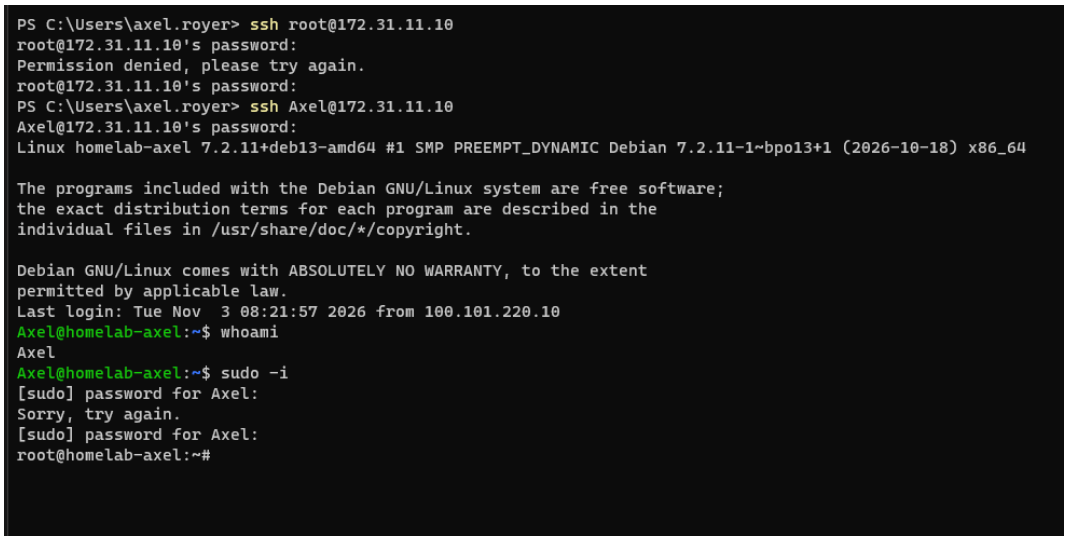


***Figure 21** — Connexion root refusée, puis connexion nominative d'Axel et élévation par sudo -i (un premier mot de passe refusé avant le succès)*

Traçabilité. Le journal de `sudo` (`journalctl _COMM=sudo`) enregistre qui a lancé quelle commande, depuis quel terminal et quel dossier. Les lignes antérieures, ouvertes par `root` lui-même, ne permettent pas d'identifier un responsable : c'est l'intérêt d'un compte nominatif.

```
root@homelab-axel:~# journalctl _COMM=sudo -n 5 --no-pager
Oct 30 14:46:12 homelab-axel sudo[32148]: pam_unix(sudo:session): session opened for user root(uid=0) by root(uid=0)
Oct 30 14:46:12 homelab-axel sudo[32148]: pam_unix(sudo:session): session closed for user root
-- Boot 79e63b90a91942f6b27a4610bf86a0eb --
Nov 02 16:38:23 homelab-axel sudo[25498]:     Axel : TTY=pts/1 ; PWD=/home/Axel ; USER=root ; COMMAND=/usr/bin/whoami
Nov 02 16:38:23 homelab-axel sudo[25498]: pam_unix(sudo:session): session opened for user root(uid=0) by Axel(uid=1000)
Nov 02 16:38:23 homelab-axel sudo[25498]: pam_unix(sudo:session): session closed for user root
root@homelab-axel:~#
```

Figure 22 — *Journal de sudo : la commande exécutée en root est tracée au nom du compte Axel*

Le secours en cas de problème reste la console Proxmox (VM 107 → Console), qui n'utilise pas SSH.

7.3 Isolation de la VM dans le tailnet

La politique d'accès Tailscale est un fichier unique, valable pour tout le tailnet. Celui-ci est le tailnet personnel de l'administrateur (homelab à la maison avec un NAS, invités partagés limités à Jellyfin) : la VM d'école l'a rejoint, et toute règle la concerne aussi le reste du réseau privé.

Risque. La VM, appareil personnel, appartient au groupe `autogroup:member`, dont la règle autorise tous les ports vers tous les appareils : elle pouvait donc ouvrir des connexions vers le NAS de la maison. Une personne disposant des droits root sur la VM, par exemple un compte jury « piloter », aurait pu utiliser l'identité Tailscale de la VM pour atteindre ce NAS.

Solution : l'étiquette tag:ecole. Un appareil étiqueté n'est plus un appareil personnel : il sort de `autogroup:member` et n'a plus aucun droit de connexion sortante. L'administrateur continue d'atteindre la VM, puisque sa règle vise tous les appareils.

Démarche sécurisée. Aucune modification n'a été enregistrée sans filet :

1. sauvegarde du texte de la politique dans un fichier (seul vrai retour arrière : un snapshot Proxmox protège la VM, pas la politique Tailscale) ;
2. ajout d'un bloc `tagOwners` et d'un bloc `tests`, les deux règles existantes restant identiques (annexe C) ;
3. contrôle par « Preview changes » : seuls des ajouts doivent apparaître ;
4. enregistrement : Tailscale contrôle la syntaxe et exécute les tests (le PC joint la VM et le NAS ; la VM étiquetée ne joint pas le NAS) avant d'appliquer ;
5. pose de l'étiquette par la console (Machines → Edit ACL tags) et non par `tailscale up --advertise-tags`, qui oblige à répéter toutes les options et risque de retirer la route annoncée.

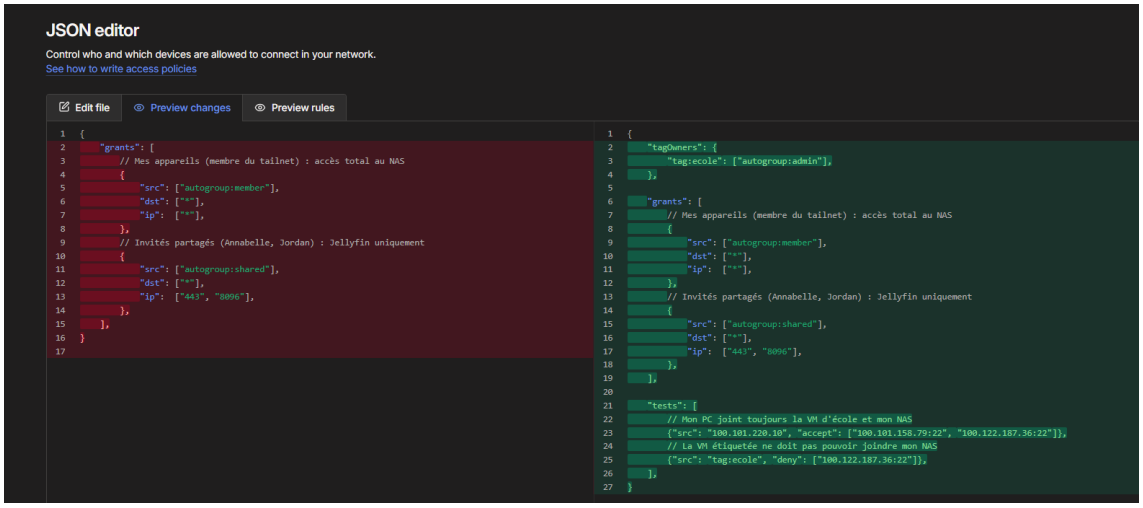


Figure 23 — *Aperçu des changements : le bloc tagOwners et le bloc tests sont ajoutés, les deux règles existantes sont reprises à l'identique*

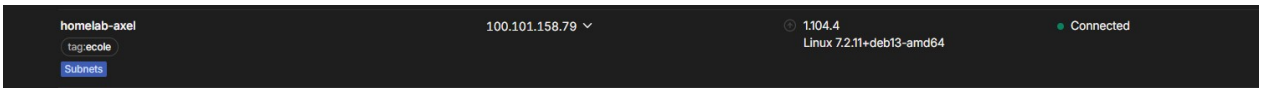


Figure 24 — La machine homelab-axel : étiquette tag:ecole, route annoncée (Subnets), client Tailscale 1.104.4, état Connected

Preuve avant / après. Depuis la VM, une tentative de connexion TCP vers le port 80 du NAS : `OUVERT` avant l'étiquette, `BLOQUE` après. Depuis le poste d'administration, `Test-NetConnection 172.31.11.10 -Port 22` continue de réussir.

```
root@homelab-axel:~# timeout 5 bash -c '</dev/tcp/100.122.187.36/80' && echo OUVERT || echo BLOQUE
OUVERT
root@homelab-axel:~# timeout 5 bash -c '</dev/tcp/100.122.187.36/80' && echo OUVERT || echo BLOQUE
BLOQUE
```

Figure 25 — Même test depuis la VM : OUVERT avant l'étiquette, BLOQUE après

Limite : cette isolation empêche la VM d'initier des connexions vers le tailnet ; elle ne limite pas les ports que les appareils de l'administrateur atteignent sur la VM.

7.4 Accès du jury (à finaliser)

Le jury peut avoir besoin d'un accès direct à l'infrastructure, par deux chemins distincts : vers **Proxmox**, géré par l'école (VPN existant, éventuel rôle en lecture seule limité au pool `axel.r`), et vers **la VM**, par un partage Tailscale de la machine. Un nœud partagé n'expose que cette machine, sans les routes annoncées ; le jury joint la VM par son nom complet ou son adresse Tailscale, et les ports accessibles sont restreints par une règle d'accès sur l'adresse du destinataire.

Tableau 15 — Niveaux d'accès prévus pour l'épreuve

Niveau	Compte	Droits
Voir et tester	`jury` (créé)	SSH sans `sudo` ni `docker`, utilisateur Jellyfin non administrateur, Homepage sans connexion ; OMV, Portainer et Kuma présentés par le candidat
Piloter	`jury-admin` (à créer)	`sudo` complet (équivalent de `root`), administrateur Jellyfin, identifiants administrateur d'OMV, Portainer et Kuma remis le jour de l'épreuve

Le niveau « piloter » n'est activé que pour l'épreuve : snapshot `AVANT_JURY` juste avant pour pouvoir restaurer la VM, puis, après l'épreuve, révocation du partage, retrait des règles d'accès du jury et changement des mots de passe. Cette partie est reportée : elle dépend de la réponse de l'école (le jury se connecte-t-il lui-même, par quel VPN accède-t-il à Proxmox, le VPN route-t-il le VLAN 11).

8. Sauvegarde et restauration

8.1 Snapshots Proxmox

Un snapshot est pris à la fin de chaque phase validée, avant d'attaquer la suivante, **sans la RAM** : un état disque propre suffit, les services reviennent seuls grâce à `restart: unless-stopped`. À chaque nouveau snapshot, l'ancien est supprimé pour limiter l'espace consommé dans le pool.

Un snapshot est stocké au même endroit que les disques de la VM (`local-lvm` de pve-1) : il offre un retour arrière rapide, mais **n'est pas une sauvegarde**, car il disparaît avec la VM si le stockage tombe, consomme de l'espace au fil des écritures et bloque la migration de la VM.

Tableau 16 — Snapshots de la VM 107

Nom	Date	Contenu	État
`P3_jellyfin_ok`	2 nov. 2026, 11:11:49	Fin de la phase 3 : Jellyfin et partage SMB	Supprimé
`P4_supervision_ok`	2 nov. 2026, 15:19:25	Fin de la phase 4 : supervision et Homepage	Supprimé
`P5_https_ok`	2 nov. 2026	Fin de la phase 5 : Jellyfin en HTTPS	Supprimé
`P6_ssh_ok`	—	Comptes `Axel` et `jury`, root SSH désactivé	Supprimé
`P6_isolation_ok`	3 nov. 2026, 09:45:31	Phase 6 : compte nominatif et sudo, root SSH désactivé, compte `jury`, mot de passe `admin` d'OMV, OMV en HTTPS, isolation de la VM par étiquette	Présent
`P7_sauvegarde_ok`	3 nov. 2026, 11:41:52	Phase 7 validée : sauvegarde vzdump et restauration testée	Présent (le plus récent)

Le journal de création du dernier snapshot montre la méthode : l'agent QEMU fige brièvement le système de fichiers de la VM (`freeze guest filesystem`), Proxmox crée un volume logique de snapshot par disque (`snap_vm-107-disk-0_P7_sauvegarde_ok` et `snap_vm-107-disk-1_P7_sauvegarde_ok`), puis le système de fichiers est libéré (`thaw guest filesystem`). La colonne « Mémoire » indique `Non` : la RAM n'est pas incluse.

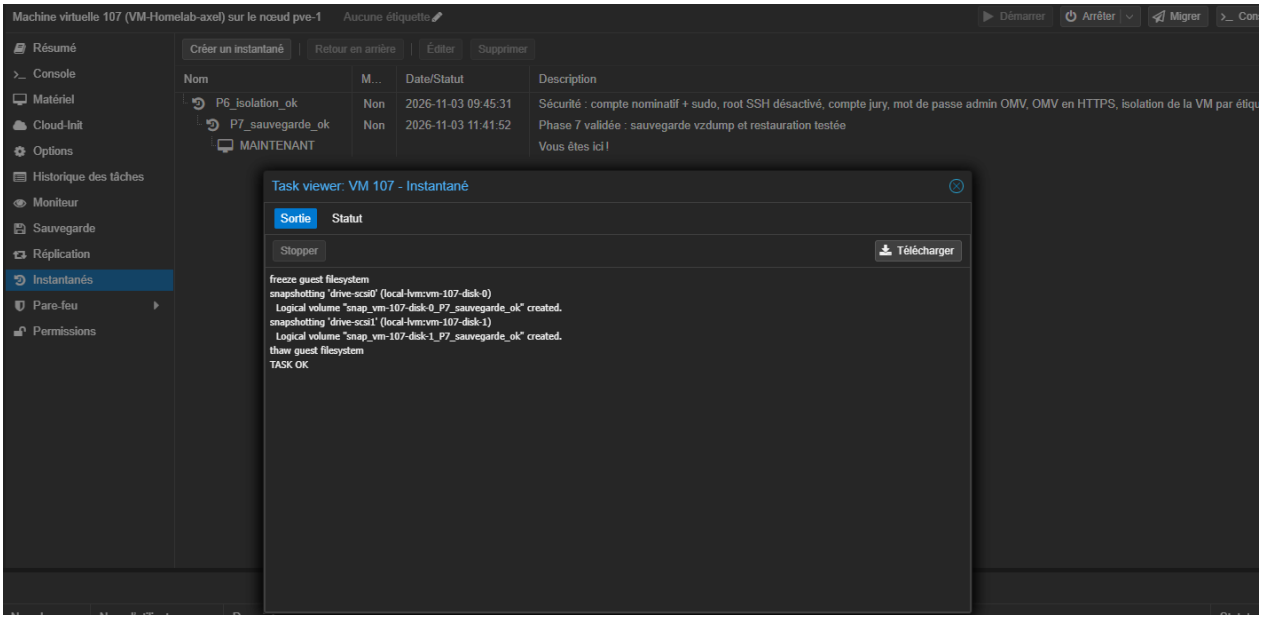


***Figure 26 —** Instantanés de la VM 107 (P6_isolation_ok et P7_sauvegarde_ok) et journal de création de P7_sauvegarde_ok*

8.2 Sauvegarde vzdump

Une sauvegarde `vzdump` écrit une archive sur un stockage séparé, `qnap` : elle survit à une panne du disque ou du nœud, et permet de recréer la VM. Les deux outils se complètent.

Tableau 17 — Snapshot et sauvegarde : deux outils complémentaires

Critère	Snapshot	Sauvegarde vzdump
Emplacement	Même stockage que la VM (`local-lvm`)	Archive sur `qnap`, stockage séparé
Panne du disque ou du nœud	Perdu avec la VM	Survit : la VM peut être recréée depuis l'archive
Usage	Retour arrière rapide après une manipulation ratée	Restauration complète après un sinistre
Déclenchement	Manuel, un par phase validée	Automatique, chaque semaine
Contenu	État du disque à un instant donné	Disques et configuration ; les snapshots n'y sont pas inclus

Constat de départ. La liste « Invités sans tâche de sauvegarde » montrait la VM 107 : aucune tâche ne la couvrait. L'école a une tâche générale (pool `Infra114`, tous les jours à 21:00, rétention `keep-daily=7`, `keep-monthly=1`…), qui ne concerne pas la VM : la nouvelle tâche ne fait donc pas doublon.

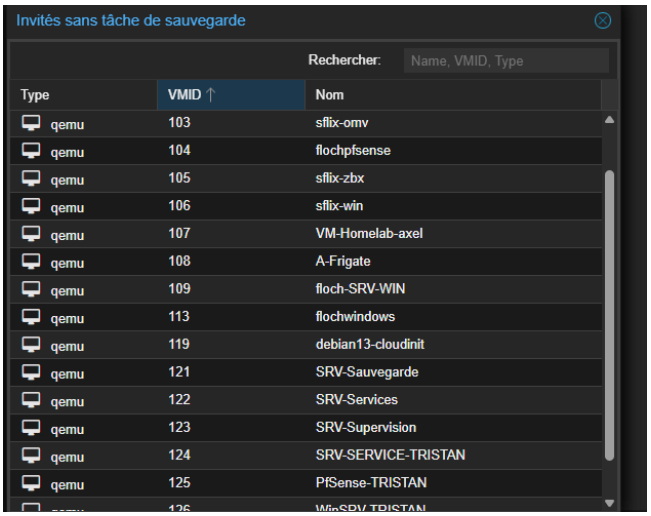


Figure 27 — Liste « Invités sans tâche de sauvegarde » : la VM 107 n'est couverte par aucune tâche

Tableau 18 — Tâche de sauvegarde de la VM 107

Paramètre	Valeur	Raison
Nœud	`pve-1`	Nœud qui héberge la VM
Stockage	`qnap`	Stockage séparé du disque de la VM
Planification	`sun 01:00`	Une fois par semaine, la nuit
Sélection	VM 107 uniquement	Les autres VM de la baie ne sont pas concernées
Mode	Instantané	La VM reste en service
Compression	ZSTD	Bon compromis entre vitesse et taille
Rétention	`keep-last = 1`	Une seule archive sur un stockage partagé avec la baie

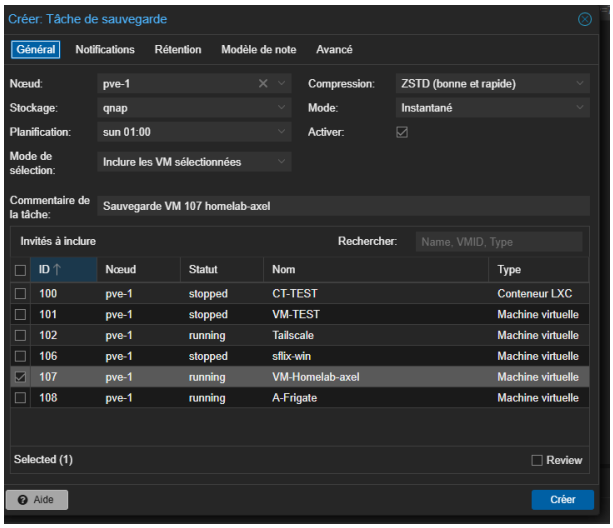


Figure 28 — Création de la tâche : nœud pve-1, stockage qnap, planification sun 01:00, mode instantané, ZSTD, VM 107

Rétention. Proxmox n'écrase jamais une archive : chaque sauvegarde crée un nouveau fichier daté, puis la rétention supprime les plus anciens une fois la nouvelle sauvegarde réussie. `keep-last` compte des sauvegardes, pas des semaines. Si une sauvegarde échoue, l'ancienne reste intacte. Limites : aucun historique (les points de retour par phase étant couverts par les snapshots) et il faut de la place pour deux archives pendant le passage d'une sauvegarde à l'autre. Sans valeur de rétention, Proxmox appliquerait celle du stockage ou du fichier `vzdump.conf` du nœud, ce qui ferait perdre la maîtrise de la règle.

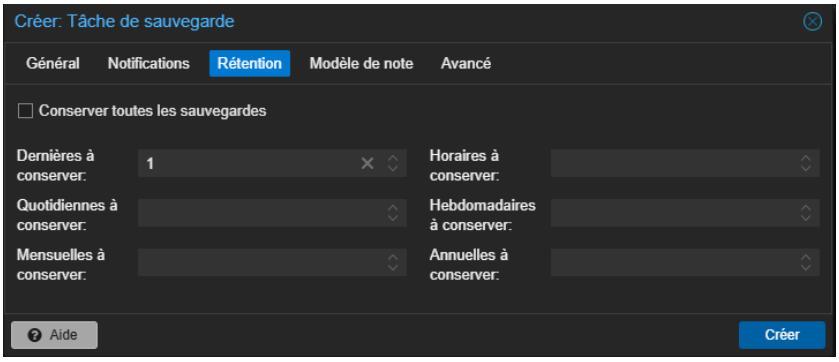


Figure 29 — Onglet Rétention : « Dernières à conserver » = 1, tous les autres champs vides

> **Points de vigilance à la création.** Une première version du formulaire avait le stockage `local` (le disque du nœud, ce qui annulait l'intérêt d'une sauvegarde) et une planification vide (champ obligatoire). Les deux ont été corrigés avant de créer la tâche.

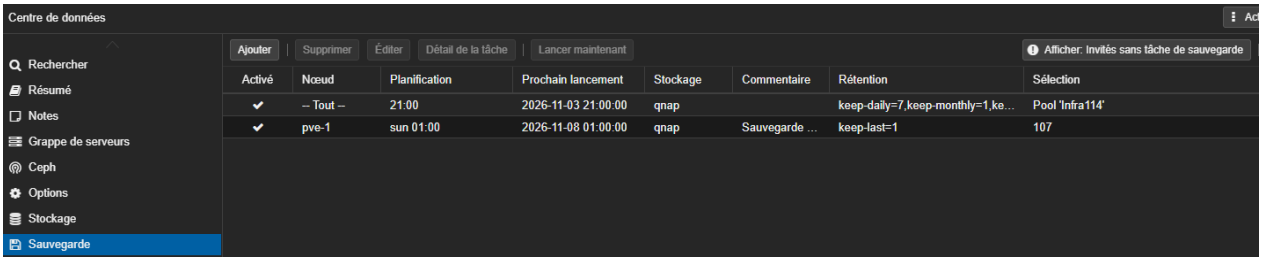


Figure 30 — Tâches de sauvegarde : tâche générale de l'école (pool Infra114) et tâche de la VM 107

Premier lancement (« Lancer maintenant », 3 novembre 2026, 10:07 à 10:14) : 7 minutes 43 secondes, sans arrêt de la VM.

Tableau 19 — Mesures du premier lancement

Mesure	Valeur
Données parcourues	84 Gio (disques de 20 Go et 64 Go)
Zones vides (sparse)	74,45 Gio, soit 88 %
Taille de l'archive	4,59 Go
Rétention	`keep-last=1` appliquée, aucune archive supprimée (c'est la première)
Résultat	`Backup job finished successfully`, `TASK OK`

Le début du journal confirme que l'agent QEMU a été utilisé : Proxmox demande à la VM de figer son système de fichiers (`fs-freeze`), lance la sauvegarde, puis libère le système de fichiers (`fs-thaw`) et la VM reprend. La copie est cohérente et la VM n'est figée que quelques instants. Le journal indique aussi `snapshots found (not included into backup)` : une VM restaurée n'aura que l'état courant des disques.

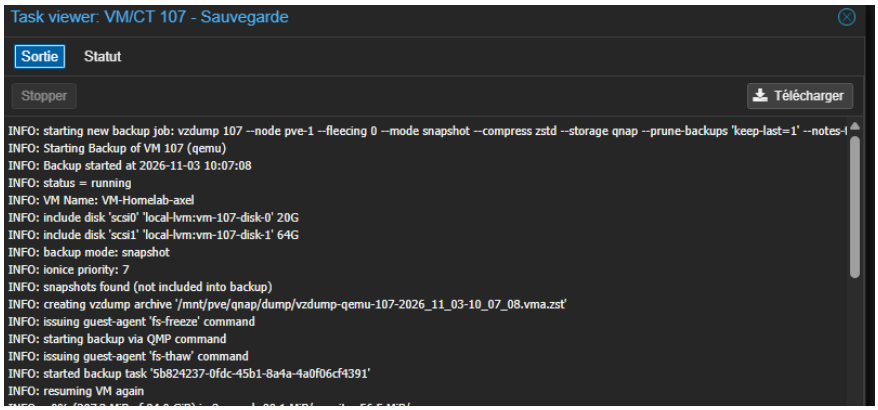


Figure 31 — *Début du journal : mode snapshot, ZSTD, stockage qnap, fs-freeze puis fs-thaw*

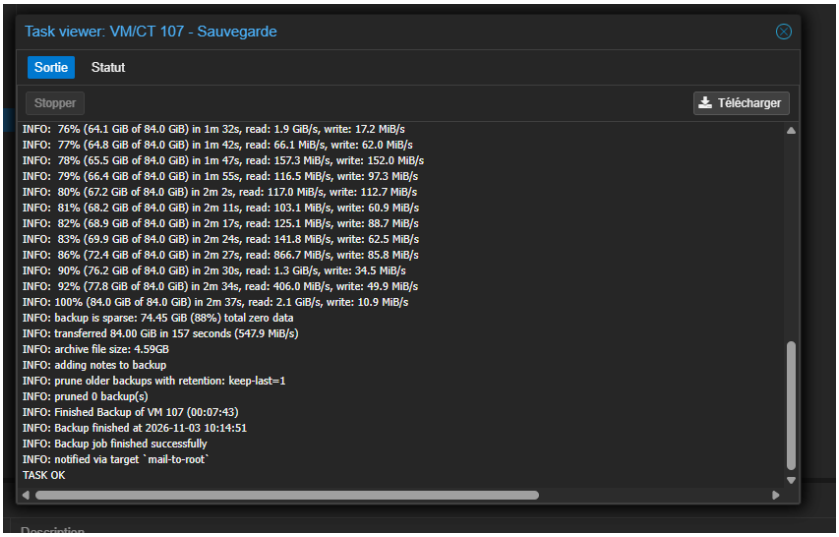


Figure 32 — *Fin du journal : 84 Gio transférés, archive de 4,59 Go, rétention appliquée, TASK OK*

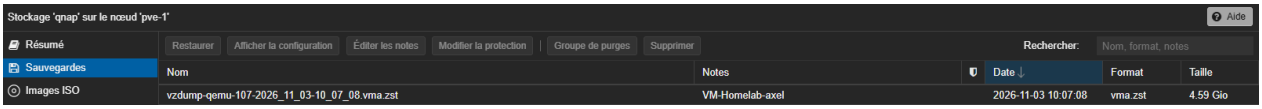


Figure 33 — *Archive présente sur le stockage qnap*

8.3 Test de restauration

Une sauvegarde jamais restaurée n'est qu'une hypothèse. Le test vérifie que l'archive est lisible et qu'elle permet de recréer une VM avec ses disques et sa configuration, sans toucher à la VM en service.

Tableau 20 — Précautions de la restauration

Précaution	Raison
Restaurer vers un nouvel identifiant (`207`)	Restaurer vers `107` écraserait la VM en service
Renommer la copie `TEST-restauration-207`	Éviter de se tromper de ligne lors de la suppression
Cocher « Unique »	Génère de nouvelles propriétés uniques, dont une nouvelle adresse MAC
Ne pas démarrer après la restauration	Une copie en réseau reprendrait l'IP 172.31.11.10 et l'identité Tailscale de la VM réelle
Vérifier l'identifiant et la place disponible	Identifiant 207 libre ; `local-lvm` avec de la place (70 Go utilisés sur 1,8 To)

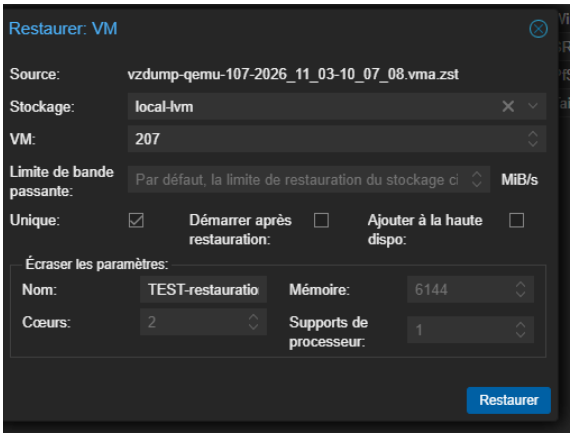


Figure 34 — Fenêtre de restauration : VM 207, stockage local-lvm, « Unique » coché, démarrage décoché, nom TEST-restauration

Résultat. Restauration terminée par `TASK OK` en 440 secondes (7 min 20 s) : 90 194 313 216 octets lus (84 Gio) dont 88,6 % de zones vides ignorées, puis `rescan volumes`.

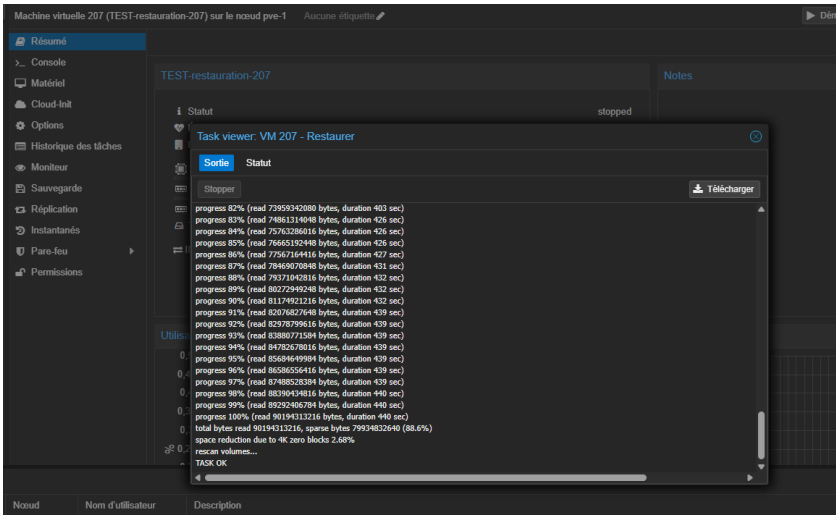


Figure 35 — Fin du journal de restauration de la VM 207 : 100 %, 440 secondes, TASK OK

Tableau 21 — Comparaison de la VM d'origine et de la copie restaurée

Contrôle	VM 107 (réelle)	VM 207 (restaurée)
Disques	20 Go et 64 Go	20 Go et 64 Go
Mémoire et processeurs	6 Gio, 2 cœurs	6 Gio, 2 cœurs
Carte réseau	`vmbr1`, VLAN 11, pare-feu	`vmbr1`, VLAN 11, pare-feu

Contrôle	VM 107 (réelle)	VM 207 (restaurée)
Adresse MAC	`BC:24:11:04:C7:08`	`BC:24:11:81:F1:3B` (différente : « Unique » a fonctionné)
Démarrage à l'amorçage	—	Non
Agent QEMU	—	Activé

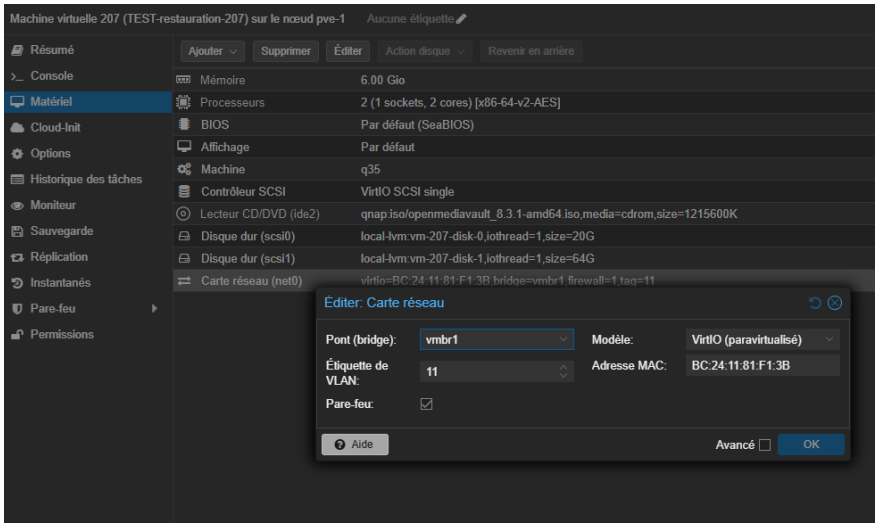


Figure 36 — Matériel de la VM 207 restaurée : mêmes disques et même mémoire, adresse MAC différente

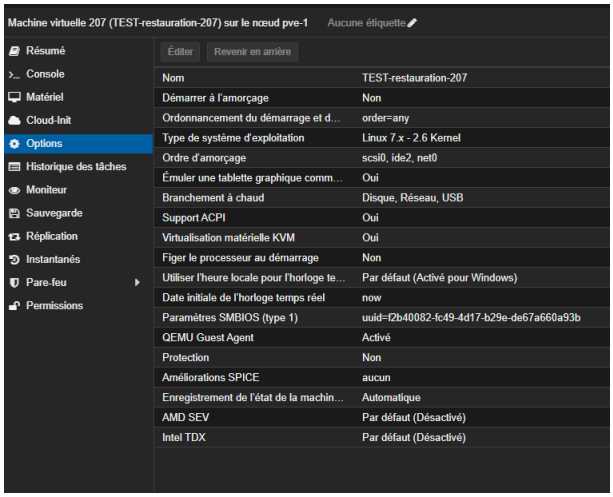


Figure 37 — Options de la VM 207 : démarrage à l'amorçage désactivé, agent QEMU activé

Démarrage de la copie : non testé, volontairement. Une copie dont la carte réseau est connectée reprendrait l'IP 172.31.11.10 et l'identité Tailscale de la VM réelle, ce qui provoquerait un conflit sur le réseau de l'école et dans le tailnet. La procédure sûre, non exécutée ici, consisterait à supprimer la carte réseau de la VM 207 avant de l'allumer, à observer le démarrage à la console, puis à l'éteindre.

Conclusion et limite. Le test prouve que l'archive est lisible et que Proxmox recrée une VM avec ses deux disques, sa configuration et une adresse MAC propre ; il ne prouve pas que le système démarre depuis la copie. Dans un vrai sinistre, la restauration se ferait vers l'identifiant 107, la VM d'origine ayant disparu. La VM de test a ensuite été supprimée (disques non référencés compris) : la VM 107 et l'archive sur `qnap` n'ont pas été touchées.

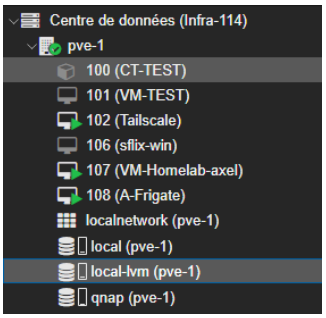


Figure 38 — *Arbre Proxmox après la suppression de la VM de test : la VM 107 est toujours présente*

8.4 Objectifs de reprise (RPO et RTO)

Deux notions cadrent la reprise après sinistre : le **RPO** (perte de données maximale acceptable) et le **RTO** (durée maximale de remise en service).

Tableau 22 — Objectifs de reprise de la VM

Notion	Valeur actuelle	Justification et limites
RPO	Jusqu'à 7 jours	Sauvegarde hebdomadaire (dimanche 01:00). Les données concernées sont surtout des configurations de services et des films, remplaçables. Les snapshots couvrent les manipulations en cours, mais pas une panne du stockage.
RTO (restauration)	7 min 20 s mesurées	Durée mesurée de la restauration vers une VM de test ; elle peut varier avec la charge du stockage partagé.
RTO (reprise complète)	Non mesuré	Le démarrage de la copie et la vérification des services n'ont pas été testés (voir 8.3).

Pistes d'amélioration. Passer à une sauvegarde quotidienne ramènerait le RPO à 24 heures pour environ 4,6 Go d'archive supplémentaire par jour conservé. Un test de démarrage de la copie, carte réseau déconnectée, permettrait de mesurer le RTO complet.

Scénario de sinistre. Si la VM 107 était détruite : restaurer l'archive la plus récente vers l'identifiant 107, démarrer la VM, vérifier `sudo docker ps`, `sudo tailscale serve status` et l'état dans Uptime Kuma, puis contrôler l'accès par l'adresse 172.31.11.10. L'identité Tailscale de la VM restaurée n'a pas été vérifiée : c'est un point à tester.

9. Exploitation

9.1 Procédures courantes

Arrêt propre. Dans Proxmox : VM 107 → Arrêter (passe par l'agent QEMU) ; ou en SSH `sudo shutdown -h now`. L'arrêt forcé est réservé à une VM bloquée.

Vérifications après un redémarrage (la VM démarre sur le noyau 7.2.11) :

```
ssh Axel@172.31.11.10
uname -r                      # noyau en cours
uptime -p                     # durée depuis le dernier démarrage
sudo docker ps                # 4 conteneurs Up : homepage, uptime-kuma, jellyfin, portainer
sudo tailscale serve status   # publications 8443 et 9444 (tailnet only)
```

Les conteneurs reviennent seuls (`restart: unless-stopped`) et les publications Serve persistent (`--bg`). Pour l'utilisation depuis le réseau de l'école, couper Tailscale sur le poste ; à distance, Tailscale doit être actif.

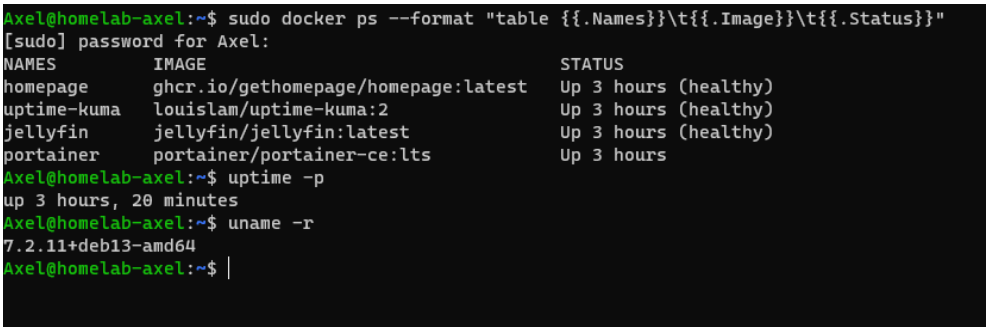


Figure 39 — Vérification sur la VM : quatre conteneurs Up (healthy), VM en fonctionnement depuis 3 h 20 min, noyau 7.2.11

9.2 Dépannage rapide

Tableau 23 — Symptômes fréquents

Symptôme	Cause probable	Action
`Permission denied` en SSH avec `root`	Connexion root désactivée (voulu)	Se connecter avec `Axel`, puis `sudo -i`
`Sorry, try again` avec `sudo`	Mot de passe de `root` saisi au lieu de celui d'`Axel`	Retaper le mot de passe d'`Axel`, lentement
`permission denied` sur `docker ps`	Compte hors du groupe `docker` (voulu)	`sudo docker ps`
Interface non joignable à distance	Tailscale inactif sur le poste ou route non approuvée	Activer Tailscale ; vérifier la route dans la console
Tuile Homepage sans point vert	Conteneur arrêté ou sonde en échec	`sudo docker ps`, journaux du conteneur
Erreur YAML dans Homepage	Contenu doublé ou mal indenté	`cat -A fichier.yaml`, vider puis recoller
Conteneur absent après enregistrement d'un fichier Compose	Bouton Up non cliqué	Sélectionner le fichier, puis Up
Accès par Tailscale perdu	VM éteinte, Tailscale arrêté sur la VM ou sur le poste, route non approuvée	Contrôler la VM (console Proxmox) et `sudo tailscale status`, puis la route dans la console

9.3 Mises à jour

Tableau 24 — Politique de mise à jour

Élément	Mode actuel	Procédure
Système (Debian, OpenMediaVault)	Mises à jour de sécurité automatiques (`unattended-upgrade`), noyau issu des dépôts backports (7.2.11)	Redémarrer la VM pour appliquer un nouveau noyau, puis contrôler `uname -r`
Images Docker	Étiquettes `latest` (Jellyfin, Homepage), `lts` (Portainer), 2 (Uptime Kuma) ; mise à jour manuelle	Snapshot, puis `docker compose pull` et `up` depuis OMV (Compose → Fichiers), puis vérification
Client Tailscale	Installé avec le script officiel	Mise à jour avec `apt` ; contrôler `tailscale version`
Proxmox et stockage	Géré par l'école	Hors périmètre

Procédure type (mensuelle, ou immédiate pour une faille de sécurité) :

1. Prendre un snapshot de la VM, sans la RAM, et supprimer l'ancien.
2. Appliquer les mises à jour (système et images).
3. Redémarrer la VM si le noyau a changé.
4. Vérifier : `sudo docker ps`, `sudo tailscale serve status`, sondes d'Uptime Kuma, tuiles de Homepage.
5. En cas d'échec, revenir au snapshot depuis Proxmox (VM 107 → Instantanés → Retour en arrière).

10. Incidents rencontrés et enseignements

Les incidents sont décrits dans les encadrés des sections concernées (Portainer en 4.2, Jellyfin en 4.3, Homepage en 4.4, comptes en 7.1). Le tableau ci-dessous reprend les plus instructifs.

Tableau 25 — Incidents principaux

Incident	Cause	Correction
Verrou apt pendant l'installation de Docker	`unattended-upgrade` mettait à jour le noyau	Attendre la libération du verrou, ne jamais le supprimer à la main
VM injoignable depuis le poste	Un partage Tailscale ne donne pas accès aux sous-réseaux	Tailscale sur la VM, route `/32` annoncée (5.1)
`ssh admin@…` refusé	`admin` est un compte de l'interface OMV, pas un compte Linux	Utiliser un compte système (`Axel`)
Lecture Jellyfin en erreur	Films de 0 octet	`ls -lh`, puis `curl -fL` (4.3)
Deux comptes `Axel` et `axel`	Casse des noms sous Linux	Lister les comptes avant d'agir (7.1)
Politique Tailscale invalide	Le mot `json` du bloc de code a été copié	Contrôler avec « Preview changes » (7.3)
Stockage `local` proposé pour la sauvegarde	Valeur par défaut du formulaire	Choisir `qnap`, stockage séparé (8.2)

Enseignements. Vérifier avant d'agir (listes de comptes, tailles de fichiers, identifiants) ; garder une session de secours ouverte lors des changements d'accès ; prendre une copie de la configuration avant de modifier une politique ; documenter chaque incident avec sa cause.

11. Tests de validation

Tableau 26 — Matrice de validation

Test	Résultat attendu	Résultat
Redémarrage de la VM	4 conteneurs relancés, publications Serve conservées	Validé
Panne de Jellyfin (arrêt du conteneur)	Alerte Down puis Up dans le salon Discord	Validé (20 s d'indisponibilité)
Panne de Portainer et d'OMV	Alertes Down puis Up	Validé (monit relance nginx)
Connexion SSH en root	Refusée	Validé
Connexion `Axel` puis `sudo -i`	Administration possible, tracée au nom d'`Axel`	Validé
`sudo -l` avec `jury`	Refusé	Validé
Certificat HTTPS des interfaces	Let's Encrypt, valide, sans alerte du navigateur	Validé (jusqu'au 31 janvier 2027)
Connexion de la VM étiquetée vers le NAS	Bloquée par la politique d'accès	Validé (OUVERT avant, BLOQUE après)
Accès du poste à 172.31.11.10 après étiquetage	Fonctionne par Tailscale	Validé
Sauvegarde vzdump de la VM en service	`TASK OK`, archive sur `qnap`	Validé (4,59 Go, 7 min 43 s)
Restauration vers une VM de test	VM recréée avec ses disques et une MAC différente	Validé (7 min 20 s)
Démarrage de la VM restaurée	Système démarré depuis la copie	Non testé (voulu)

12. Scénario de démonstration (10 minutes)

Ce scénario enchaîne les preuves les plus parlantes. Il suppose la VM allumée et un poste d'administration avec Tailscale actif (à l'école, l'adresse 172.31.11.10 fonctionne aussi bien en direct).

Tableau 27 — Déroulé de la démonstration

#	Durée	Action	Ce qu'on montre	Résultat attendu
1	1 min	Présenter le schéma (figure 1), puis ouvrir Homepage : `http://172.31.11.10:3000`	Architecture, services et état	Tuiles avec points verts
2	2 min	`ssh root@172.31.11.10`, puis `ssh Axel@172.31.11.10` et `sudo -i`, puis `journalctl _COMM=sudo -n 3 --no-pager`	Root refusé, compte nominatif, traçabilité	`Permission denied` ; invite `root@homelab-axel` ; ligne `Axel : … COMMAND=…`
3	2 min	`sudo docker stop jellyfin`, patienter, puis `sudo docker start jellyfin`	Panne, alerte, rétablissement	Alerte Down dans `#jellyfin`, puis Up avec la durée
4	1 min 30	Depuis la VM : `timeout 5 bash -c '</dev/tcp/100.122.187.36/80' && echo OUVERT || echo BLOQUE` ; console Tailscale, étiquette `tag:ecole`	Isolation de la VM	`BLOQUE`
5	1 min	Ouvrir `https://homelab-axel.tail14f35b.ts.net:9444` et afficher le certificat	HTTPS et certificat	Page OMV sans alerte, émetteur Let's Encrypt
6	2 min 30	Proxmox : tâche de sauvegarde, archive sur `qnap`, fenêtre de restauration (VM 207, « Unique », démarrage décoché) sans la lancer, journal de la restauration d'essai	Sauvegarde séparée et restaurable	Archive de 4,59 Go ; 7 min 20 s mesurées

Préparation, la veille

- VM allumée ; vérifier `sudo docker ps`, les sondes d'Uptime Kuma et la réception des alertes dans Discord.
- Avant l'étape 3, régler la sonde Jellyfin sur un intervalle de 20 s sans nouvelle tentative pour que l'alerte arrive en moins d'une minute, puis remettre 60 s et 2 tentatives.
- Si le jury doit piloter la VM : prendre le snapshot `AVANT_JURY` juste avant la session.
- Plan B : la console Proxmox (VM 107 → Console) et les captures de ce document si un accès réseau échoue.

Questions probables

Tableau 28 — Questions du jury et réponses courtes

Question	Réponse courte	Section
Pourquoi une sauvegarde en plus des snapshots ?	Le snapshot est sur le même stockage que la VM et disparaît avec elle ; la sauvegarde est sur `qnap`.	8.1, 8.2
Que se passe-t-il si la VM tombe ?	Uptime Kuma tombe avec elle et ne peut pas alerter : limite assumée, la supervision devrait être sur une autre machine.	6, 13
Pourquoi désactiver le root en SSH ?	Son nom est connu de tout attaquant ; un compte nominatif apporte la traçabilité.	7.2

Question	Réponse courte	Section
Pourquoi Tailscale plutôt qu'un VPN classique ?	Aucun port à ouvrir sur le réseau de l'école ; ACL et certificats HTTPS inclus.	2.2, 5
Qu'est-ce qui n'a pas été testé ?	Le démarrage de la VM restaurée, pour ne pas dupliquer l'IP et l'identité Tailscale.	8.3, 8.4
Quelle perte de données en cas de sinistre ?	Jusqu'à 7 jours (sauvegarde hebdomadaire).	8.4

13. Limites et perspectives

Limites connues

- Uptime Kuma est hébergé sur la VM qu'il surveille : une panne de la VM n'est pas signalée.
- Le stockage `qnap` est géré par l'école et partagé avec la baie : la sauvegarde n'est pas hors site, et la rétention d'une seule archive ne conserve aucun historique.
- La supervision ne mesure que la disponibilité des services (pas l'espace disque ni la mémoire) ; les notifications de sauvegarde de Proxmox sont envoyées à `mail-to-root`, qui n'est pas relayé.
- Le démarrage d'une VM restaurée n'a pas été testé, pour ne pas dupliquer l'adresse IP et l'identité Tailscale.
- L'accès HTTP de repli à l'école reste en clair ; il protège le service pour les postes sans Tailscale mais n'est pas chiffré.
- La clé Tailscale de la VM n'expire plus : en cas de compromission, la VM doit être retirée du tailnet à la main ; l'étiquette `tag:ecole` limite ce qu'elle peut atteindre.
- L'accès du jury (comptes de pilotage, partage Tailscale, règles d'accès) dépend de la réponse de l'école.

Perspectives

- **Phase 8 : Bookaru.** Hébergement de l'application (Caddy, dépôt git) et exposition publique par Tailscale Funnel sur le port 443, avec une modification de la politique d'accès (attribut `funnel`) suivant la même démarche sécurisée.
- **Accès du jury.** Création du compte « piloter », partage de la VM et règles d'accès dédiées, avec snapshot `AVANT_JURY` et révocation après l'épreuve.
- **Détection.** Un projet distinct pourrait déployer Wazuh (SIEM) sur une seconde VM pour détecter les tentatives d'intrusion et mesurer le durcissement.
- **Supervision externalisée** et copie hors site de la sauvegarde.

Annexe A. Fichiers Docker Compose

Les chemins `CHANGE_TO_COMPOSE_DATA_PATH` sont remplacés par le plugin Compose d'OMV par le dossier partagé `dockerdata`. `<UUID>` désigne l'identifiant du disque de données.

Portainer

```
services:
  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - CHANGE_TO_COMPOSE_DATA_PATH/portainer:/data
```

Jellyfin

```
services:
  jellyfin:
    image: jellyfin/jellyfin:latest
    container_name: jellyfin
    restart: unless-stopped
    ports:
      - "8096:8096"
    volumes:
      - CHANGE_TO_COMPOSE_DATA_PATH/jellyfin/config:/config
      - CHANGE_TO_COMPOSE_DATA_PATH/jellyfin/cache:/cache
      - /srv/dev-disk-by-uuid-<UUID>/films:/media/films:ro
```

Uptime Kuma

```
services:
  uptime-kuma:
    image: louislam/uptime-kuma:2
    container_name: uptime-kuma
    restart: unless-stopped
    ports:
      - "3001:3001"
    volumes:
      - CHANGE_TO_COMPOSE_DATA_PATH/uptime-kuma:/app/data
```

Homepage

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      HOMEPAGE_ALLOWED_HOSTS: homelab-axel:3000,172.31.11.10:3000,100.101.158.79:3000
    volumes:
      - CHANGE_TO_COMPOSE_DATA_PATH/homepage:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
```

Annexe B. Configuration de Homepage (services.yaml)

```
- Infrastructure:
    - OpenMediaVault:
        icon: openmediavault.png
        href: https://homelab-axel.tail14f35b.ts.net:9444
        description: Administration du NAS (HTTPS, via Tailscale)
        siteMonitor: http://172.31.11.10
    - OpenMediaVault (réseau école):
        icon: openmediavault.png
        href: http://172.31.11.10
        description: Administration du NAS (HTTP, réseau de l'école)
        siteMonitor: http://172.31.11.10
    - Portainer:
        icon: portainer.png
        href: https://172.31.11.10:9443
        description: Gestion des conteneurs
        server: local
        container: portainer
    - Uptime Kuma:
        icon: uptime-kuma.png
        href: http://172.31.11.10:3001
        description: Supervision et alertes
        server: local
        container: uptime-kuma

- Médias:
    - Jellyfin:
        icon: jellyfin.png
        href: https://homelab-axel.tail14f35b.ts.net:8443
        description: Films (HTTPS, via Tailscale)
        server: local
        container: jellyfin
        siteMonitor: http://172.31.11.10:8096/health
    - Jellyfin (réseau école):
        icon: jellyfin.png
        href: http://172.31.11.10:8096
        description: Films (HTTP, réseau de l'école)
        siteMonitor: http://172.31.11.10:8096/health
```

Annexe C. Politique d'accès Tailscale

Politique finale du tailnet. Les deux règles d'origine sont inchangées ; le bloc `tagOwners` déclare l'étiquette et le bloc `tests` sert de garde-fou à l'enregistrement. Les adresses sont celles du poste d'administration, du NAS personnel et de la VM.

```
{
    "tagOwners": {
        "tag:ecole": ["autogroup:admin"],
    },

    "grants": [
        // Appareils personnels (membres du tailnet) : accès total
        {
            "src": ["autogroup:member"],
            "dst": ["*"],
            "ip":  ["*"],
        },
        // Invités partagés : Jellyfin uniquement
        {
            "src": ["autogroup:shared"],
            "dst": ["*"],
            "ip":  ["443", "8096"],
        },
    ],

    "tests": [
        // Le PC joint toujours la VM d'école et le NAS
        {"src": "100.101.220.10", "accept": ["100.101.158.79:22", "100.122.187.36:22"]},
        // La VM étiquetée ne doit pas pouvoir joindre le NAS
        {"src": "tag:ecole", "deny": ["100.122.187.36:22"]},
    ],
}
```

Annexe D. Commandes utiles

Tableau 29 — Commandes d'exploitation

Objet	Commande
Conteneurs en cours	`sudo docker ps`
Journal d'un conteneur	`sudo docker logs <nom> --tail 50`
Racine de Docker	`docker info | grep "Docker Root Dir"`
Adresse Tailscale de la VM	`tailscale ip -4`
Publications HTTPS	`sudo tailscale serve status`
Annuler une publication	`sudo tailscale serve --https=9444 off`
Configuration SSH chargée	`sudo sshd -T | grep -i permitrootlogin`
Journal des commandes sudo	`journalctl _COMM=sudo -n 5 --no-pager`
Mémoire disponible	`free -h` (colonne « available »)
Fichier YAML (caractères cachés)	`cat -A fichier.yaml`

Objet	Commande
Suspendre la surveillance de nginx (test de panne)	`monit unmonitor nginx` puis `monit monitor nginx`
Réinitialiser le mot de passe `admin` d'OMV	`omv-firstaid` depuis la console Proxmox